

Technical University of Applied Sciences Würzburg-Schweinfurt (THWS)
Faculty of Computer Science and Business Information Systems

Master Thesis

Great Research About My Favourite Topic

submitted for the completion of degree

**Master of Science
in
Artificial Intelligence**

Your Name

Submitted on: March 10, 2026

Supervisor: Prof. Dr. Magda Gregorova
Second Examiner: Prof. Dr. C D

Abstract (en)

A single paragraph summarizing the most important points of your thesis. This can be more than one paragraph but should not be too long. You usually write this as the very last thing of your thesis.

Abstract (de)

We are in germany so include a translation.

Acknowledgment

Here you can thank your friends who helped you throughout the studies, your parents for supporting you generally, if you wish your supervisor or any other profs and generally everybody you value.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research Problem	2
1.3	Thesis Objective	3
1.4	Research Goals	4
1.5	Research Questions	4
2	Systematic Literature Review	6
2.1	Phase 1 — Planning	6
2.1.1	Review Protocol	6
2.2	Phase 2 — Conducting the Review	13
2.2.1	Execution of the Search Strategy	13
2.2.2	Removing of Duplications	13
2.2.3	Study Selection	13
3	Background	15
3.1	Vector Graphics Fundamentals	15
3.1.1	Paths and Bézier Curves	16
3.1.2	Coordinate Systems and Transformations	16
3.1.3	Scope of SVG in This Thesis	16
3.2	Vector Representations	16
3.2.1	Parametric Representation	17
3.2.2	Sequential Representation	17
3.2.3	Implicit Neural Representation	17
3.2.4	Latent Representation	18
3.3	Differentiable Rendering	18
3.4	Loss Functions and Semantic Guidance	19
3.4.1	Reconstruction Losses	19
3.4.2	CLIP	19
3.4.3	Diffusion Models	20
3.4.4	Score Distillation Sampling and VSD	20
4	Architectural Landscape of SVG Generation	22
4.1	Optimization-Based Methods	23
4.2	Dataset-Driven Methods	25

5	Vector Representation in SVG Generation	28
5.1	Parametric Representations	29
5.2	Sequential Representations	29
5.3	Implicit Neural Representations	30
5.4	Latent Representations	31
5.5	Hybrid Representations	32
5.6	Representation and Editability	33
5.7	Structural Properties Emerging from Representation	33
5.8	Summary	34
6	Supervision and Guidance Strategies for SVG Generation	36
6.1	Reconstruction-Based Supervision	37
6.2	Semantic Guidance with Vision–Language Models	37
6.3	Diffusion-Based Supervision	38
6.4	Dataset Supervision vs. Per-Instance Supervision	39
6.5	Structural Priors and Initialization as Supervision	40
6.6	Summary	41
7	Evaluation	42
7.1	Evaluation of Visual Quality	43
7.2	Evaluation of Structural Quality	44
7.3	Editability	46
7.4	Efficiency and Practical Usability	47
7.5	Quantitative vs Qualitative Evaluation Gap	49
7.6	Toward a Multi-Dimensional Framework for Evaluating SVG Quality . .	50
8	Comparative synthesis	53
8.1	Structure Patterns	53
8.2	Supervision $\rightarrow$ Geometry Behavior	54
8.3	Editability versus Visual Realism Trade-off	56
8.4	Complexity vs Control: Computational Implications	58
8.5	Structurally Clean and Editable SVG Generation	59
8.6	Design Implications for Future SVG Generation	60
9	Conclusion	62
	Bibliography	63
	List of Figures	67
	Appendix	68
	Great table.	68
	Table of chosen approaches	69

Declaration on oath	79
Consent to plagiarism check	80

1 Introduction

Vector graphics differ fundamentally from raster images. Raster images represent visual content as grids of pixels, where each pixel stores color information without explicit semantic meaning. In contrast, vector graphics describe images using mathematical primitives such as lines, curves, shapes, and groups. Because these primitives are defined parametrically, vector graphics can be scaled, transformed, and edited without loss of quality.

Among vector formats, Scalable Vector Graphics (SVG) has become a widely adopted standard. SVG is a text-based representation that encodes geometry, styling, and hierarchical relationships between graphic elements. As a structured format, SVG allows graphics to be programmatically generated and modified, which makes it particularly suitable for computational design workflows.

From the perspective of generative modeling, these structural properties introduce additional challenges. Unlike raster images, SVG graphics explicitly encode geometry and topology. As a result, generating SVG graphics requires more than visual similarity. Generated outputs must also maintain structural correctness so that they remain interpretable. Graphics that contain redundant paths, broken hierarchy, or unstable geometry may appear visually plausible while being difficult to reuse in practical settings.

Before learning-based approaches, vector graphics were primarily produced through algorithmic vectorization techniques that approximate raster images using geometric primitives. Although such methods remain widely used, they rely on heuristic rules and often struggle with complex scenes, stylized content, or semantic guidance from text. The emergence of learning-based approaches introduces new possibilities but also raises questions about how models construct vector structure internally.

1.1 Motivation

In recent years, generative models have significantly transformed the landscape of visual design and digital content creation. Diffusion models, large-scale multimodal systems such as CLIP, and transformer-based architectures have demonstrated that high-quality

image synthesis from text is no longer a research prototype but a practical reality. Systems such as Stable Diffusion, DALL-E, and Midjourney are capable of producing visually convincing raster images that are sometimes difficult to distinguish from human-created content. Continuous improvements in realism, controllability, and semantic alignment have accelerated the adoption of generative models in creative workflows.

However, this progress has been much more pronounced in the raster image domain than in vector graphics. While research on vector graphics generation has grown rapidly, the reliability, controllability, and structural quality of generated vector graphics remain open challenges. At the same time, vector graphics continue to play a central role in modern digital design. Icons, logos, illustrations, user interface elements, and data visualizations are typically created and maintained in vector formats because they must remain editable and resolution independent.

This importance is reflected not only in research but also in industry. Design tools such as Adobe Illustrator and Figma rely heavily on vector representations as the core format for interactive editing. Recent developments in design copilots and AI-assisted creative tools further highlight a shift toward generation systems that must produce editable graphics rather than static images. Emerging integrations between generative models and design platforms suggest that future workflows will require generation methods capable of producing structurally meaningful vector graphics that can be directly reused and modified by designers.

These trends motivate the study of SVG generation not only as a generative modeling task but also as a representation problem, where the structure of the output is as important as its visual appearance.

1.2 Research Problem

Research on SVG generation has expanded across multiple directions, including sequence-based models, optimization-driven approaches, neural representations, and multimodal generation pipelines. These methods differ significantly in their internal representations, the supervision signals used during training or optimization, and the mechanisms through which these signals are translated into final SVG outputs. Representation choices influence optimization stability, geometric consistency, and editability, yet these relationships remain insufficiently understood.

A central difficulty is that many works emphasize visual similarity between generated outputs and reference images while structural properties of SVG graphics receive less systematic analysis. Generated graphics may achieve strong perceptual quality but still exhibit issues such as fragmented geometry, redundant paths, missing hierarchy,

or limited editability. This creates a gap between visually convincing generation and practically usable vector graphics.

In addition, evaluation practices for SVG generation remain inconsistent. Quantitative metrics commonly adapted from raster image synthesis, such as FID or reconstruction-based losses, primarily measure visual similarity rather than structural quality. While these metrics provide useful signals, they do not capture properties such as geometric consistency, topology, hierarchy, or editability. Consequently, quantitative results reported across different models may not fully reflect practical SVG quality, making comparison between approaches difficult.

Furthermore, the diversity of internal representations — including parametric primitives, command sequences, latent embeddings, and neural implicit functions — complicates understanding how architectural design influences structural outcomes. There is currently no unified perspective that explains how representation and optimization strategies relate to the resulting vector structure.

This lack of systematic analysis of representation, structure, and evaluation constitutes the central research gap addressed in this thesis.

1.3 Thesis Objective

This thesis investigates SVG generation and vectorization methods from the perspective of representation, optimization, and structural quality. Rather than proposing a new generation model, the goal is to develop a structured understanding of how existing architectures construct vector graphics and how these design choices influence the resulting SVG structure.

Particular emphasis is placed on models whose generation process remains interpretable in terms of vector operations or clearly defined architectural components. While recent large language model-based approaches treat SVG generation as structured code synthesis, their internal generation process is less explicit with respect to geometric construction. For this reason, language-model-based SVG generation is considered primarily as an emerging direction, while the main analysis focuses on optimization-based, neural, and hybrid approaches where representation choices can be directly examined.

To support systematic analysis, the thesis focuses on a selected set of open-source SVG generation models with available implementations and published descriptions. This enables comparative study across methods that differ in representation strategy, supervision signals, and optimization pipelines. The analysis aims to clarify how different approaches balance visual fidelity with structural correctness and to identify evaluation

criteria relevant to real-world design workflows.

1.4 Research Goals

1. To explain and analyze the core properties, structures, and components of SVG graphics as a foundation for understanding their representation and generation.
2. To collect, review, and categorize existing open-source methods for SVG generation across various input modalities.
3. To investigate the types of deep learning and machine learning architectures employed in SVG generation models and analyze their design principles.
4. To examine how these architectures represent SVG data internally and how their outputs are transformed into standard SVG files.
5. To explore and evaluate existing approaches, metrics, and benchmarks for assessing the quality of generated SVGs.
6. To assess and compare the output quality of current SVG generation models based on the identified evaluation criteria.

1.5 Research Questions

1. What are the main properties and structures that define an editable and high-quality SVG?
2. What open-source methods currently exist for generating SVGs from different input modalities?
3. What types of deep learning and machine learning architectures are models used for SVG generation?
4. What kinds of structure do these models produce?
5. How can the quality and editability of generated SVG be evaluated?
6. What is the observed quality of SVG outputs across different models?

The remainder of this thesis builds on the conceptual progression introduced above. The background chapter introduces the fundamental concepts required to understand vector graphics generation, including SVG structure, internal representation types, differentiable rendering, and supervision strategies used in learning-based methods.

Building on this foundation, subsequent chapters review existing SVG generation approaches and analyze how architectural design choices influence vector structure. Particular attention is given to the relationship between representation strategy and the properties of generated graphics.

Following the architectural analysis, the thesis introduces evaluation perspectives for SVG generation that extend beyond visual similarity. Both quantitative and qualitative criteria are considered, including measures related to editability, geometric consistency, and practical usability.

These evaluation perspectives are then applied in a comparative analysis of selected open-source models, where strengths, limitations, and trade-offs between approaches are examined. The thesis concludes by summarizing key insights, discussing limitations, and outlining directions for future research in structured vector graphics generation.

2 Systematic Literature Review

Kitchenham’s guidelines were originally developed for evidence-based software engineering and have become one of the most widely adopted standards for conducting systematic literature reviews in computer science and technical research disciplines [kitchenham’procedures’2004]. The methodology provides detailed, domain-specific procedures for planning, searching, selecting, and synthesizing studies in areas where research outputs are often heterogeneous—such as algorithmic models, machine learning architectures, or engineering pipelines. Unlike biomedical-focused protocols such as PRISMA [moher’preferred’2009], which were designed primarily for clinical trials and human-subject research with standardized quantitative outcomes, Kitchenham addresses challenges unique to computational research: diversity of study designs, inconsistent terminology, non-uniform evaluation metrics, and the frequent presence of code-based or algorithmic artifacts instead of comparable statistical datasets. Similarly, frameworks like SALSA [noauthor’typology’nodate] emphasize high-level structuring but do not prescribe the step-by-step operational procedures needed for reproducibility in technical fields.

For this thesis—situated within deep learning, machine learning, and vector-graphics generation—a Kitchenham-style SLR is needed because the literature consists of diverse model architectures, varied input modalities, and different evaluation criteria. Kitchenham’s protocol supports this complexity by providing structured phases (planning, conducting, and reporting) that guide the reviewer through formulating rigorous research questions, defining search strategies, conducting selection criteria, assessing quality, data extraction and performing narrative synthesis. This structure minimizes bias, increases reproducibility, and ensures that the review captures the full range of relevant deep-learning and vectorization approaches.

2.1 Phase 1 — Planning

2.1.1 Review Protocol

Databases

- arXiv
- Google Scholar
- ACM Digital Library

Search Strings/Keywords

SVG Generation

"SVG generation"
"scalable vector graphics" AND generation
"SVG generative models"
"vector graphics generation"
"SVG generation" AND deep learning
"SVG generation" AND machine learning

Raster-to-Vector Vectorization

"raster-to-vector" AND deep learning
"image-to-vector" AND neural network
"image vectorization" AND machine learning
"deep vectorization"
"differentiable vector graphics"

Text-to-SVG

"text-to-SVG"
"text-to-vector graphics"

Structure of SVG, Editability and Quality

"SVG Basics"
"SVG fundamentals"
"Scalable Vector Graphics"

"SVG editability"
"editable SVG"
"SVG path simplification"
"layered SVG" AND generation"

Survey and Review Searches

"SVG" AND "systematic literature review"
"vector graphics" AND survey
"vector image" AND review
"image vectorization" AND survey
"graphic generation" AND review
"SVG Generation" AND benchmark
"image vectorization" AND benchmark

Study Selection Criteria

Inclusion Criteria:

- **Language and Publication:** Written in English and published in a peer-reviewed journal, conference, or workshop.
- **Time Range:** Published between 2020–2025 (post-emergence of modern SVG generative models such as DeepSVG).
- **Domain Relevance:** Falls within computer vision, computer graphics, AI, machine learning, or deep learning.
- **Reproducibility:** Provides an open-source implementation or references a publicly available repository. Availability of pre-trained weights is preferred, but source code alone is sufficient if the method is reproducible.
- **SVG Output Requirement:** Describes a model, method, or architecture that directly generates SVG or structured vector outputs, including:
 - Bézier curves,
 - SVG `<path>` commands,
 - layered or hierarchical SVG structure,

- vector strokes or SVG-like stroke sequences,
- structured outputs suitable for editability analysis.
- **Input Modalities:** Supports at least one of:
 - image-to-SVG vectorization,
 - text-to-SVG generation,
 - multimodal SVG generation,
- **Architectural Transparency:** Provides sufficient technical detail to extract the model architecture and internal SVG representation (e.g., decoder design, transformer layers, vector tokenization, differentiable rasterization, or geometric parameterization).
- **Evaluation:** Provides quantitative metrics and qualitative SVG examples that allow analysis of structural quality and editability.

Exclusion Criteria:

- **No Vector Output:** The method does not generate vector or SVG outputs (pixel-only image generation).
- **Non-ML Methods:** Purely rule-based or classical algorithmic vectorization methods without machine learning or deep learning.
- **Closed-Source Systems:** No publicly available code, weights or sufficient technical detail for reproduction.
- **Insufficient Evaluation:** Papers lacking SVG evaluation, qualitative vector examples or structural analysis of outputs.
- **Out-of-Scope Vector Work:** Studies focused on vector editing, rendering, compression, markup optimization, or stylistic modification without SVG generation.
- **Domain Mismatch:**
 - handwriting synthesis,
 - font generation,

- datasets,
- sketch generation not intended for SVG graphics.
- **Temporal Exclusion:** Studies published before 2020, except when cited for background context only.
- **LLM Restriction:** Exclude proprietary LLMs that generate SVG text but do not provide architecture, training details, or reproducibility.

Study Selection Procedure

- Initial records
- Duplicate removal
- Title/abstract screening
- Full-text assessment
- Final included studies

Quality Assessment Criteria

Following the recommendations by Kitchenham for evidence-based software engineering [**kitchenham`procedures`2004**, **kitchenham`systematic`2009**], each primary study must be assessed for quality, the assessment criteria must be explicitly described, applied consistently, and the overall quality of each study should be reported (e.g., high, medium, or low quality). The four Kitchenham categories used in this review are reporting quality, methodological rigor, reproducibility, and result validity. The criteria are listed below.

The following Kitchenham-style quality criteria were applied to all studies that passed the full-text screening. Each criterion was evaluated as **Yes = 1** or **No = 0**. The total score was then used to classify studies as high, medium, or low quality.

Reporting Quality

- The study states its research aim clearly.
- The model or method architecture is described in enough detail.

- The SVG/vector representation (paths, strokes, Béziers, etc.) is explained.

Methodological Rigor

- The method is appropriate for SVG/vector generation.
- The evaluation procedure is described clearly.

Reproducibility

- The study provides open-source code or enough detail to reproduce results.

Result Validity

- Quantitative metrics or qualitative SVG examples are provided.

Scoring and Classification Each criterion was scored as either **Yes = 1** or **No = 0**. The total score determined the study's quality classification:

- **6–7 points:** High quality
- **4–5 points:** Medium quality
- **0–3 points:** Low quality (may be excluded)

Kitchenham recommends that “studies should be classified as high, medium or low quality based on quality criteria” [**kitchenham'procedures'2004**].

Data Extraction Plan According to Kitchenham's guidelines for evidence-based software engineering, the purpose of data extraction is to systematically collect all information required to answer the research questions in a consistent and reproducible way. Kitchenham emphasises that a predefined data extraction form must be created *during the planning phase* to ensure uniformity and reduce subjective variation among reviewers [**kitchenham'procedures'2004**].

Predefined Data Extraction Form Kitchenham states that researchers should develop a structured data extraction form before conducting the review. This prevents ad-hoc decisions and ensures that all necessary information is collected consistently across all studies [kitchenham'procedures'2004].

Alignment with Research Questions Kitchenham notes that extracted data must be driven directly by the review's research questions. The form should therefore include only fields that support answering the research questions or performing the planned synthesis [kitchenham'procedures'2004].

Information to Extract The guidelines specify that extraction should capture both bibliographic information and structured study attributes, including study context, aims, methodology, data used, experimental setup, and results. Kitchenham emphasises that the level of detail must be sufficient for later comparison and synthesis [kitchenham'procedures'2004].

Reviewer Consistency Kitchenham recommends that extracted data should be validated—ideally by a second reviewer—to minimise bias and ensure consistency. If this is not possible, reviewers should re-check their own extraction for consistency [kitchenham'procedures'2004].

Structured (Not Free-Form) Fields Kitchenham advises the use of structured or semi-structured fields to minimise subjectivity and ensure that comparable information is extracted across studies. Examples include fixed fields for model architecture, datasets, evaluation metrics, and SVG output structure [kitchenham'procedures'2004].

Avoiding Unnecessary Information Only data needed for answering the research questions or for synthesis should be extracted. Kitchenham warns that recording irrelevant details increases effort and contributes no value to the final review [kitchenham'systematic'2009].

Ensuring Comparability Across Studies Finally, Kitchenham highlights that the extraction process must ensure that the information gathered from different studies is comparable and suitable for aggregation or narrative synthesis [kitchenham'systematic'2009]. Using a consistent extraction form enables systematic comparison of SVG generation models.

2.2 Phase 2 — Conducting the Review

2.2.1 Execution of the Search Strategy

The search strings defined in Phase 1 were executed across the selected databases (Google Scholar, arXiv, and ACM Digital Library) to identify relevant literature on SVG generation and vector-based deep learning methods. Searches were performed using the conceptual string groups defined in the search strategy, with database-specific syntax adjustments where necessary. All records retrieved by these searches were exported and documented before any filtering or removal of duplicates. Table 2.1 summarises the number of studies returned by each search group and database.

Table 2.1: Search Log Summary Across Databases and Conceptual Groups

Search Group	Google Scholar	arXiv	ACM DL	Total
SVG Generation	14	12	4	30
Raster-to-Vector	7	10	2	19
Text-to-SVG	7	6	5	18
Surveys/Benchmarks/Reviews	6	3	1	10
SVG Fundamentals / Background	3	3	2	8
Total Initial findings	37	34	14	85

2.2.2 Removing of Duplications

After merging all search results, duplicate records—including repeated titles and preprint–conference duplicates—were removed. This resulted in 72 unique studies.

2.2.3 Study Selection

- Stage 1: Title, Keywords and Abstract screening.

After removing duplicates, **72** unique studies remained. A brief title and abstract screening excluded works clearly unrelated to SVG generation (e.g., sketch-only models, raster methods, classical vectorization, or closed-source LLMs), resulting in **37** papers proceeding to full-text review.

- Stage 2: Full-text screening

A full-text assessment was conducted on the **37** studies that passed the title and abstract screening. Each paper was examined for reproducibility, architectural transparency, and whether it genuinely produced SVG or structured vector outputs as defined by the inclusion criteria. In total, **10** studies were excluded at this stage: four lacked any open-source implementation, two were benchmark papers evaluating proprietary LLMs such as GPT, Gemini, or Claude, two used non-reproducible LLM-based pipelines without sufficient methodological detail, and one relied on a purely rule-based vectorization approach. Following this eligibility check, **27** studies remained for quality assessment.

Table 2.2: Reasons for Exclusion During Full-Text Screening

Exclusion Reason	Count
No open-source implementation	4
Proprietary LLM benchmark studies (GPT, Gemini, Claude)	2
Non-reproducible LLM-based pipeline	2
Purely rule-based vectorization method	2
Total Excluded	10

Table 2.3: Study Selection Summary Across Screening Stages

Selection Stage	Number of Studies
Initial search results (all databases)	85
After duplicate removal	72
After title and abstract screening	37
After full-text screening	27

3 Background

The models analyzed in this thesis draw on a shared set of technical foundations: the structure of SVG itself, the representations used to manipulate it during learning, the rendering bridge that connects vector parameters to pixel space, and the supervision signals that drive optimization. Understanding these components is what makes it possible to compare models not just by their outputs, but by the structural choices that produce them.

3.1 Vector Graphics Fundamentals

Scalable Vector Graphics (SVG) is an XML-based standard for representing two-dimensional graphics as structured text [1]. Unlike raster formats, an SVG file is a readable document that describes how an image should be drawn rather than storing pixel values. At its core, an SVG file consists of instructions that define geometric primitives, styling, and hierarchical relationships between elements [2].

SVG supports a range of graphic elements, including basic shapes such as lines, rectangles, and circles, as well as text and raster image embeds. The most expressive element is the path, which allows complex shapes to be defined using sequences of commands: moving a virtual pen, drawing straight lines, and creating smooth curves [1].

SVG also supports grouping and layered organization of elements. Multiple shapes can be combined into groups that behave as a single unit for styling and editing [2]. Elements are rendered in a defined order, and later elements appear on top of earlier ones, making layering an intrinsic part of the representation. This hierarchical structure matters for generation: models that produce only geometric approximations without respecting layer organization generate outputs that are visually plausible but difficult to edit.

3.1.1 Paths and Bézier Curves

Most SVG graphics are built using paths. A path is a sequence of commands — *move to*, *line to*, *curve to* — that define its shape [1]. Curved segments use Bézier curves, which are smooth curves controlled by a set of control points that influence curvature without lying directly on the curve.

SVG supports both quadratic Bézier curves, which use a single control point, and cubic Bézier curves, which use two and offer greater flexibility for complex shapes [1]. From the perspective of SVG generation, these control points are the core geometric parameters that most models either predict directly or optimize iteratively.

3.1.2 Coordinate Systems and Transformations

SVG graphics are drawn within a coordinate system defined by a viewport, with the origin at the top-left corner, the x-axis pointing right, and the y-axis pointing downward [2]. Transformations — translation, scaling, rotation, skewing — can be applied to individual shapes or to groups, allowing collections of paths to be moved or resized while preserving their internal structure. Nested groups define local coordinate frames, which is particularly relevant for layered compositions where subgroups must maintain their relative geometry independently.

3.1.3 Scope of SVG in This Thesis

SVG includes additional features such as clipping, masking, and filter effects. These primarily affect rendering appearance rather than geometric structure, and are not considered further here. This thesis treats SVG as a structured representation of geometric primitives, paths, and hierarchical relationships. Understanding these components is what allows the generation models analyzed in Chapters 4 through 8 to be compared on structural grounds — not just by their visual output.

3.2 Vector Representations

SVG generation and vectorization models do not all manipulate vector graphics the same way internally. The final output is always an SVG file — text-based, editable, conforming to the same standard. What differs is the internal representation used during learning or optimization, and this choice directly affects which algorithms can be applied, how stable

training is, and what structural properties tend to emerge. The following subsections introduce the four representation types that appear across the models analyzed in this thesis.

3.2.1 Parametric Representation

SVG files are already parametric by nature: every element is defined by a set of numeric values — control point coordinates, stroke widths, colors — that can be read, modified, and written as plain text [1]. Parametric representations in generation models follow this same structure directly. A graphic is treated as a collection of primitives, each fully described by continuous parameters that can be adjusted independently. In optimization-based methods, generation works by initializing a set of shapes and iteratively refining their parameters until the rendered output matches a target [3].

3.2.2 Sequential Representation

A different way to think about vector graphics is as a sequence of drawing instructions: move the pen here, draw a line there, curve to this point. This mirrors how SVG path data is actually written, but it also opens up a connection to language modelling — a vector graphic encoded as a command sequence looks structurally similar to a sentence [4]. Command order determines both drawing sequence and layering, and autoregressive models can generate graphics token by token the same way they generate text. The trade-off is that errors accumulate: a mistake early in a long sequence can corrupt everything that follows.

3.2.3 Implicit Neural Representation

Implicit representations take a fundamentally different approach: there are no explicit shapes, no stored control points, no command sequences. Instead, a neural network defines the graphic entirely through its weights [5]. Given a spatial coordinate as input, the network outputs properties at that location — color, layer assignment, shape membership. The geometry is never stored; it emerges from evaluating this continuous function across space. Because the entire representation is differentiable by construction, implicit models integrate naturally with gradient-based optimization and diffusion-based supervision.

3.2.4 Latent Representation

Latent representations are not vector representations in the geometric sense. Rather than describing the structure of a single graphic, a latent space compresses many graphics into a shared numerical embedding learned during training. A VAE, for example, encodes inputs into compact vectors and learns to reconstruct them through a decoder [6]. The actual geometry — paths, control points, commands — still lives in one of the other representation types underneath. This concept is included here because several models in Chapter 5 use latent spaces as an organizational layer on top of their base representation, and that distinction matters when comparing how their outputs are structured.

3.3 Differentiable Rendering

Before neural networks can optimize vector graphics using image-based loss functions, there needs to be a way to connect the two spaces. Vector parameters — control points, stroke widths, colors — live in a continuous geometric space. Loss functions operate on pixel values. These two spaces are not naturally connected: traditional rasterization converts vector geometry into pixels, but this process is not differentiable, meaning gradients cannot flow back through it to update the vector parameters.

DiffVG, introduced by Li et al. [7], solves this by approximating rasterization in a way that supports gradient computation. Given a set of vector parameters as input, DiffVG renders a pixel image as output. Because the rendering step is differentiable, a loss computed on the output image — whether a pixel reconstruction loss, a perceptual loss, or a signal from a pretrained semantic model — can be backpropagated all the way through to the vector parameters themselves. The system learns which control point to move, and in which direction, to reduce the error. Technically, DiffVG handles the discrete visibility changes and anti-aliasing effects that make standard rasterization non-differentiable [7].

Before DiffVG, optimizing vector graphics with gradient-based methods was not straightforward. Its introduction made it possible to apply the full range of image-based supervision strategies — reconstruction losses, CLIP guidance, diffusion priors — directly to vector parameters. Most of the optimization-based models analyzed in this thesis rely on this capability, which is why it is introduced here as a foundational component rather than a model-specific detail.

3.4 Loss Functions and Semantic Guidance

Even when a model manipulates vector parameters internally, comparing outputs in pixel space is more practical than comparing vector structures directly. Most visual data exists as raster images, and pixel-based comparison gives access to well-established loss functions from computer vision. When combined with differentiable rendering, gradients from these losses flow back through the rasterizer to update vector parameters. This section introduces the main supervision signals used across the models analyzed in this thesis.

3.4.1 Reconstruction Losses

The simplest losses operate directly on pixel values. The L2 loss measures the mean squared difference between predicted and target pixels, while the L1 loss measures the mean absolute difference [8]. Both are computationally efficient and provide stable gradients when a reference image is available. Their limitation is that they treat each pixel independently, without capturing higher-level structure such as shape boundaries or object regions.

Perceptual losses address this by comparing feature representations extracted from pre-trained convolutional networks rather than raw pixel values. The formulation proposed by Johnson et al. [9] uses intermediate feature maps from VGG [10], which capture edges, textures, and object-level structure. Comparing at the feature level produces supervision signals that better reflect visual similarity as humans perceive it, and helps preserve overall structure even when exact pixel alignment differs slightly.

For tasks where shape boundary accuracy matters more than color fidelity, overlap-based losses are used. Intersection over Union (IoU) [11] measures the overlap between predicted and target regions. In vector graphics, mask-based supervision of this kind encourages correct topology and spatial consistency across layers, rather than minimizing pixel intensity differences.

All three loss types depend on having a reference raster image. In text-to-SVG settings, where no target image exists, alternative guidance strategies are needed.

3.4.2 CLIP

CLIP (Contrastive Language–Image Pretraining), introduced by Radford et al. [12], is trained on large-scale image–text pairs to learn a shared embedding space where

semantically matching pairs are placed close together and mismatched pairs are pushed apart. Given an image and a text prompt, CLIP measures their similarity through separate image and text encoders, typically using cosine similarity between the resulting embeddings.

In SVG generation pipelines, the rendered vector graphic is passed through the CLIP image encoder while the text prompt is encoded separately. If the rendered output does not match the prompt semantically, the similarity score will be low. Because CLIP is differentiable with respect to its image input, and because images can be produced from vector parameters via differentiable rendering, the CLIP similarity score can serve as an optimization signal. Gradients flow from the similarity score back through the rasterizer to the SVG parameters, allowing the system to adjust shapes and colors toward better semantic alignment. CLIP does not generate images in this setup — it acts as a semantic evaluator that provides a direction for improvement.

3.4.3 Diffusion Models

Diffusion models are generative models that learn to reverse a gradual noise corruption process. During training, clean images are progressively corrupted by adding noise, and the model learns to denoise them step by step. After training, new images can be generated by starting from random noise and iteratively denoising [13]. Stable Diffusion [14] extends this by performing the diffusion process in a compressed latent space rather than directly in pixel space, significantly reducing computational cost while preserving image quality. When conditioned on text, diffusion models learn to generate images consistent with the textual description, capturing complex visual distributions and rich semantic structure.

3.4.4 Score Distillation Sampling and VSD

Score Distillation Sampling (SDS), introduced by DreamFusion [15] for text-to-3D generation and later adopted for vector graphics by VectorFusion [16], uses a pretrained text-conditioned diffusion model as an optimization signal. The current SVG is rendered to an image, noise is added according to the diffusion schedule, and the diffusion model predicts the denoised version conditioned on the text prompt. The difference between the noisy image and the predicted denoising defines a gradient, which is back-propagated through the rasterizer to update the SVG parameters. No ground-truth image is required — the diffusion model’s learned prior acts as the teacher.

A known limitation of SDS is oversaturation: because the gradient update treats the rendered image as if it were pure noise, results tend to be over-smoothed and lack fine

detail. Variational Score Distillation (VSD), introduced in SVGDreamer [17], addresses this by modeling the rendered output as a particle in a distribution rather than treating it as noise directly. This produces more stable gradients and visually consistent results. Both SDS and VSD are used across several models analyzed in Chapter 6.

4 Architectural Landscape of SVG Generation

SVG generation is not a single problem with a single solution. Some methods treat vector graphics as optimization targets and adjust geometric parameters until a rendered image matches a prompt or a reference. Others learn from datasets and generate SVGs through a trained decoder. A smaller group avoids explicit geometric primitives entirely and uses neural functions to define shapes indirectly. These differences are not just technical details — they reflect fundamentally different assumptions about what a good SVG output should look like, and they lead to very different trade-offs between visual quality, structural organization, and editability.

This chapter introduces the fourteen models that form the analytical core of this thesis. They were selected from a broader research landscape that also includes sequence-based sketch models, retrieval-based vectorization systems, and more recently, large language model approaches to SVG synthesis. That wider context is sketched briefly below to clarify where the selected models sit and why they were chosen. The organizing principle is simple: these fourteen models together cover the full range of design decisions — in representation and supervision — that shape how SVG structure emerges from a generation pipeline. A summary of all fourteen models is provided in Table 4.1.

Research in SVG generation spans several directions, each addressing the problem from a different starting point. An early line of work treats vector drawings as ordered sequences of commands or strokes. Inspired by language modelling, methods like SketchRNN [18] showed that neural networks can learn stroke dynamics from data. Later work extended this to full SVG command prediction, including Sketchformer [19] and related sequence models [20, 21]. These approaches established that SVG structure could be learned statistically, but long command chains make training difficult and generalization to complex scenes remains limited by the scarcity of large-scale SVG datasets.

A second direction introduced differentiable rasterization as a bridge between vector parameters and image-space supervision. DiffVG [7] made it possible to optimize vector primitives using gradients from pixel-space losses. This opened the door to reconstruction-based vectorization and, later, to semantically supervised generation using pretrained diffusion models. Most of the optimization-based models in this thesis belong to this lineage.

A third direction replaces explicit geometric primitives with neural implicit functions. Instead of optimizing Bézier coordinates directly, these methods run optimization in neural parameter space, where an MLP defines shape boundaries indirectly. This approach has connections to neural radiance fields [22] and represents an attempt to improve structural coherence through representational design rather than supervision alone.

More recently, large language models and multimodal variants have entered SVG generation. Methods such as StarVector [23], OmniSVG [24], LLM4SVG [25], and InternSVG [26] treat SVG as structured code and generate it through transformer architectures pretrained on large-scale text and image corpora. Visual alignment improves substantially compared to earlier sequence models, but structural consistency across long XML sequences remains a fundamental challenge. More practically, these models depend on large proprietary architectures, making independent reproduction and industrial deployment difficult. For this reason — and because the relationship between architectural decisions and structural outputs is difficult to trace through billions of parameters — LLM-based generation is not included in the main analysis. A more detailed justification is provided in the appendix.

4.1 Optimization-Based Methods

Optimization-based methods share a common premise: SVG primitives are not predicted by a trained model but initialized and then iteratively refined until the rendered output satisfies a supervision signal. The SVG is the variable being optimized, not the output of a forward pass. What varies across methods in this family is what that supervision signal is, and how geometry is represented during optimization.

The enabling technology across all of them is differentiable rasterization. DiffVG [7] connects vector parameter space to pixel space by making the rasterization step differentiable, as described in Chapter 3. This allows any loss defined on a rendered image to propagate gradients back to Bézier control points, stroke widths, and colors. The optimization loop is the same across all methods in this family: initialize primitives, render, compute loss, update parameters. What changes is the loss and the representation.

Reconstruction-Supervised

The most direct use of this loop is pixel reconstruction. LIVE [3] optimizes vector primitives layer by layer, building up progressively finer approximations of a target raster image. The layer-wise strategy encourages hierarchical organization and tends to produce outputs where individual paths retain interpretable roles. SAMVG [27] runs

the same core loop but uses the Segment Anything Model [28] to initialize primitives at semantically meaningful regions rather than random positions. The core difference between their outputs is almost entirely a function of initialization — which suggests that how optimization begins matters as much as how it proceeds.

Im2Vec [29] sits at the edge of this group. It trains an encoder-decoder network to predict parametric vector primitives from a raster image in a single forward pass, supervised through differentiable rendering and reconstruction loss. At inference there is no iterative refinement. Im2Vec is included here rather than in the dataset-driven section because its primary contribution is a differentiable rendering-based training objective, not a learned generative model over SVG structure.

Semantic-Supervised

The optimization loop changes character when the supervision signal comes from a pretrained diffusion model rather than a target image. Score Distillation Sampling (SDS) [15], described in Chapter 3, allows gradients derived from a text-conditioned diffusion model to update vector parameters directly. The SVG is rendered, evaluated against a text prompt, and updated to move toward the learned distribution. No paired SVG training data is needed.

VectorFusion [16] was among the first to apply this to SVG, using a latent diffusion model [14] as the supervision backbone. SVGDreamer [17] builds on this by replacing standard SDS with Vectorized Particle-based Score Distillation (VPSD), which treats the rendered output as a particle in a learned distribution rather than a noisy sample — producing more stable gradients and reducing oversaturation. It also introduces explicit control over primitive count and type, organized into semantic layers. SVG-Dreamer++ [30] extends this with hierarchical refinement mechanisms that adapt vector organization during generation.

Neural Implicit Representation

A third variant keeps the optimization loop but changes the representation. NiVeL [5] and NeuralSVG [31] define geometry through the weights of an MLP rather than through explicit Bézier coordinates. Optimization still occurs at inference time, but the parameter space is neural rather than geometric. NiVeL decomposes the scene into layered implicit components, each defined by a neural field whose zero-level set traces a shape boundary. NeuralSVG encodes scene structure through an instance-based representation, where each semantic region is a separately optimized neural component. Because geometry is produced through a shared function rather than a collection of independent

paths, both models tend toward more globally coherent outputs — though at the cost of increased inference time.

4.2 Dataset-Driven Methods

Where optimization-based methods construct each SVG from scratch at inference time, dataset-driven methods learn structural patterns from collections of vector graphics and generate through a trained forward pass. The optimization happens during training, not at inference. This changes the fundamental character of generation: outputs reflect statistical regularities in the training data rather than the iterative refinement of any single instance.

DeepSVG [4] is the foundational model in this family. It learns a hierarchical latent space over SVG path commands using a transformer-based encoder-decoder trained on the SVG-Icons dataset [4]. The latent space separates path-level and command-level representations, allowing the model to capture both coarse shape organization and fine geometric detail. At inference, SVG commands are generated autoregressively from a sampled latent vector. DeepSVG demonstrated that structured SVG generation from a learned latent space is feasible, and also exposed the difficulty of maintaining geometric validity across long command sequences.

DeepIcon [32] applies a similar framework to icon generation specifically, trained on a curated dataset of simple icons. The domain-specific training produces outputs with more consistent style and geometry than general-purpose models, though generalization beyond the training distribution is limited by design.

LayerTracer [33] approaches the problem differently. Rather than predicting SVG commands from a latent vector, it models the construction process of vector graphics layer by layer. A diffusion-based architecture generates intermediate structural representations that guide each new layer, approximating the sequential way designers build graphics in practice. This process-aware formulation produces outputs with more meaningful layer organization than direct command prediction, with direct consequences for editability.

T2V-NPR [34] combines dataset training with a learned neural path representation. Trained on natural image and stylization data, it generates vector paths through a latent path space rather than explicit Bézier coordinates. It uses diffusion-based semantic supervision during training, but at inference it operates as a forward generative model. Its placement here reflects the fact that its structural behavior — consistent paths, stable geometry, dataset-bounded style — is closer to the learned models in this section than to the per-instance optimization methods above.

SVGFusion [35] is the most architecturally complex model in this family. It first learns a continuous latent space of vector graphics using a Vector Path Variational Autoencoder (VP-VAE), trained to compress path sequences into dense latent representations. Generation then occurs through a Vector Semantic Diffusion Transformer (VS-DiT), which operates entirely in this learned latent space. This makes SVGFusion distinct from the semantic-supervised optimization methods in the previous section: where VectorFusion and SVGDreamer use diffusion gradients to guide per-instance optimization in pixel space, SVGFusion uses a diffusion process over a learned vector distribution to sample new SVGs directly. The outputs reflect the statistical structure of the training corpus rather than the convergence of any optimization loop.

Table 4.1 summarizes the fourteen models analyzed in this thesis. The *Inference* column distinguishes between methods that optimize vector parameters at inference time (*Iterative Opt.*) and methods that generate SVGs through a single trained forward pass (*Forward Dec.*). The *Supervision* column indicates the primary signal driving generation: *Opt.-based* refers to direct pixel reconstruction without a pretrained model, *Self-Sup.* to semantic signals derived from frozen pretrained models (diffusion or CLIP), and *Supervised* to training against ground-truth SVG data. Task types distinguish between *Vectorization* (raster image as input, SVG as output), *Text-to-SVG* (text prompt as input), and *Unconditional Generation* (no input at inference — SVG sampled directly from a learned latent space).

Table 4.1: Overview of the fourteen models analyzed in this thesis, organized by generation strategy.

Model	Task	Inference	Input	Output	Supervision
LIVE	Vectorization	Iterative Opt.	Image	Paths	Optimization
SAMVG	Vectorization	Iterative Opt.	Image	Paths	Optimization
Im2Vec	Vectorization	Forward Dec.	Image	Paths	Self-Sup.
VectorFusion	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup.
SVGDreamer	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup.
SVGDreamer++	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup.
NiVeL	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup.
NeuralSVG	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup.
DeepSVG	Unconditional	Forward Dec.	None	Paths	Supervised
DeepIcon	Vectorization	Forward Dec.	Image	Layers/Paths	Supervised
SVGFusion	Text-to-SVG	Forward Dec.	Text	Layers/Paths	Supervised
T2V-NPR	Text-to-SVG	Forward Dec.	Text	Layers/Paths	Supervised
LayerTracer	Text-to-SVG	Hybrid	Text/Img	Layers/Paths	Supervised

The fourteen models described in this chapter will be examined through two analytical lenses in the chapters that follow. The first is representation: how each model encodes vector geometry internally, and what that encoding makes possible or forecloses in terms

of structural organization and editability. The second is supervision: what signal drives the generation process, and how that signal shapes the geometry that emerges. These two dimensions do not operate independently — a model’s representation constrains what supervision signals are meaningful, and the choice of supervision often determines which representations are practical. Understanding how they interact is the analytical task of the remainder of this thesis.

5 Vector Representation in SVG Generation

SVG generation models differ not only in their architectures and training objectives but also in how they represent vector graphics internally during the generation process. While all approaches ultimately produce SVG files as output, the internal representation used during learning or optimization varies considerably across methods. This variation has direct consequences for how geometry is parameterized, how supervision signals are applied, and what structural properties the resulting graphics have.

In the context of this thesis, representation refers to the intermediate form in which a model encodes vector geometry, not the final output file but the internal structure through which shapes are created, manipulated, and decoded. A vector graphic can be represented in several fundamentally different ways. It can be described as a collection of explicit geometric primitives with directly optimizable parameters. It can be encoded as an ordered sequence of drawing commands that procedurally construct the image. It can be defined implicitly through the weights of a neural network that maps indices or coordinates to shape properties. Or it can be embedded into a compact latent space from which the model samples and decodes structured output. Each of these formulations encodes the same underlying geometry differently, and this difference shapes how the model learns, how gradients propagate, and what the generated SVG looks like structurally.

Representation is therefore not a neutral implementation detail. It determines whether a model can produce clean hierarchical structure, how easily the output can be edited after generation, and how well the method scales to complex visual content. These properties are central to the comparative analysis in this thesis.

This chapter examines four primary representation families identified across the selected models: parametric, sequential, implicit neural, and latent. For each family, the analysis describes how it works technically, where it originates, and what structural tendencies it introduces in SVG generation.

5.1 Parametric Representations

Parametric representations describe vector graphics as collections of explicit geometric primitives, specifically paths defined by Bézier control points, stroke attributes, and fill colors. These parameters can be optimized directly using gradient-based methods or predicted by a neural network. The representation closely matches the SVG format itself, making the internal state of the model directly interpretable in geometric terms.

The use of parametric representations in neural SVG generation became practical with the introduction of DiffVG [7], which made the rasterization of Bézier primitives differentiable. Before this, gradients could not flow from pixel-space losses back to vector parameters. DiffVG enabled models to optimize control points and colors directly against image-based or semantic objectives, which is the foundation on which LIVE [3], SAMVG [27], VectorFusion [16], and SVGDreamer [17] are all built.

In practice, these models initialize a fixed set of vector primitives and iteratively refine their parameters so that the rendered image aligns with a target signal. The initialization strategy varies: LIVE uses a layer-by-layer approach that progressively adds paths, while SVGDreamer’s SIVE module uses diffusion model attention maps to determine where primitives should be placed before optimization begins [17]. Despite these differences in initialization, all of these methods share the same underlying representational commitment, namely that shapes are stored as explicit geometric parameters that are modified directly during optimization.

The main limitation of parametric representations is that complexity is handled by increasing the number of primitives. When a model needs to capture fine visual detail, it tends to introduce many overlapping or redundant paths rather than increasing the expressiveness of individual ones. LIVE, for example, uses 160 paths across five layers to represent a single image [3]. At this scale, the output is theoretically accessible for editing since each path is a geometric object, but in practice the number and entanglement of paths makes meaningful manual editing very difficult. SVGDreamer++ explicitly identifies path entanglement as a failure mode of optimization-based parametric methods [17].

5.2 Sequential Representations

Sequential representations treat vector graphics as ordered sequences of drawing commands. Rather than storing shapes as independent geometric objects, the model generates a series of commands that describe how the graphic is constructed step by step. This mirrors the structure of SVG path data directly.

The use of sequences to represent drawings originates in work on sketch generation. Ha and Eck’s SketchRNN [18] demonstrated that recurrent neural networks could learn to generate coherent hand-drawn sketches by modeling pen stroke sequences autoregressively. This established the idea that a drawing could be treated as a variable-length sequence and modeled with the same architectures used for language. DeepSVG [4] extended this approach to structured SVG icons by combining a hierarchical VAE with a transformer decoder that generates sequences of SVG path commands. DeepIcon [32] follows the same sequential formulation, learning to generate icon SVGs from command-level supervision.

Sequential representations allow the use of powerful sequence modeling architectures and can capture drawing patterns present in large datasets. However, they have known scaling limitations. Long command sequences are difficult to model reliably, and prediction errors early in the sequence propagate through the entire graphic. More fundamentally, a procedural command sequence does not encode explicit spatial hierarchy, meaning the model generates commands in order but nothing in the representation directly encodes which commands belong to which semantic region. Editability is therefore limited even when visual reconstruction quality is high.

5.3 Implicit Neural Representations

Implicit neural representations encode vector graphics through the weights of a neural network rather than through explicit geometric parameters. Instead of storing control points or commands, the model learns a function that maps indices or coordinates to shape properties such as geometry, color, and layer assignment. Shapes are defined implicitly by evaluating this function.

This approach is directly inspired by Neural Radiance Fields (NeRF) [22], which demonstrated that a compact MLP could represent a full 3D scene implicitly by mapping spatial coordinates to color and density values. NeuralSVG explicitly acknowledges this connection: each SVG is represented as a set of indices where each index corresponds to a single shape, and a small MLP learns to map these indices to Bézier control points and fill colors [31]. The network weights are the representation, not the shapes themselves. This also allows NeuralSVG to adopt an ordered representation strategy, where shapes are assigned to fixed indices during training to ensure stable correspondence between the generated output and the supervision signal.

NIVeL follows a related paradigm, representing layered vector graphics through neural functions optimized against image-based supervision. Rather than predicting individual path parameters as separate outputs, NIVeL encodes the entire layered structure implicitly, where each layer is defined by a continuous neural function over the image

domain [5]. This enables the model to produce clean layered decompositions without requiring explicit path initialization.

Implicit representations offer strong expressive power and can produce outputs with coherent global structure because the entire graphic is encoded in a single set of network weights. The main limitation is interpretability. Individual shapes are not stored as separate accessible objects, and converting the implicit output into clean, directly editable SVG primitives requires an additional extraction step. The quality of this conversion determines how usable the output is in practice.

5.4 Latent Representations

Latent representations introduce an abstraction layer by encoding vector graphics as compact embeddings in a learned latent space. Rather than representing shapes directly, the model learns a space in which structured SVG content can be sampled, interpolated, or denoised. Generation then occurs by decoding these embeddings into explicit vector primitives or command sequences.

The foundational mechanism is the variational autoencoder (VAE) [6], which trains an encoder to compress structured data into a continuous latent space and a decoder to reconstruct it. DeepSVG [4] applies this to SVG generation by encoding path command sequences into latent vectors and training a transformer decoder to reconstruct them. The latent space captures structural regularities across the training dataset, enabling the model to sample new SVGs by decoding points sampled from this space.

Latent diffusion extends this idea by applying diffusion-based generative modeling directly in the latent space of a pretrained vector encoder [14]. SVGFusion operates in this way, treating the latent vector representation as the space in which the diffusion process runs. T2V with Neural Path Representation takes a different approach, arguing that global SVG-level latent spaces are too constrained to capture diverse path combinations, and instead learns a neural path representation at the individual path level [34].

Latent representations improve scalability and generative diversity compared to direct parametric or sequential approaches. The trade-off is that edits in latent space affect the decoded output in ways that are not always predictable at the level of individual shapes. Fine-grained geometric control is harder to achieve than with parametric representations because the relationship between latent dimensions and specific visual properties is not explicit.

5.5 Hybrid Representations

In practice, many SVG generation models combine elements from more than one representation family. These combinations reflect the fact that no single representation fully addresses the competing requirements of structural correctness, editability, scalability, and visual realism.

Optimization-based diffusion methods such as VectorFusion and SVGDreamer use parametric primitives as the direct optimization target while relying on a latent diffusion model as the semantic prior. The parametric layer provides geometric interpretability, while the latent model provides semantic coherence that pure geometric optimization cannot achieve alone. SVGFusion takes a similar combined approach, encoding path-level structure into latent embeddings before applying diffusion, effectively combining latent and sequential elements in a single pipeline.

LayerTracer [33] represents a different kind of hybrid strategy. Rather than combining representation families at inference time, it integrates them during training. The model is trained on the Serpentine dataset, which contains step-by-step human SVG construction sequences, meaning the training data encodes a sequential procedural representation. However, the output the model learns to produce is a layered parametric SVG, where each layer corresponds to a meaningful visual component. The result is that editability becomes a learned property rather than an emergent one: because the model has been trained on human drawing sequences, it learns to produce outputs that are structured the way a human designer would structure them, with clean layer separation and manageable path counts.

NeuralSVG also combines ideas across families. Its core representation is implicit, with an MLP mapping shape indices to geometric parameters, but it incorporates an ordered assignment mechanism originally introduced in DeepSVG to ensure that shape indices correspond consistently to visual regions across training [31]. This ordered structure imposes a form of explicit hierarchy on what is otherwise a purely neural representation.

These hybrid strategies highlight that representation design in SVG generation is less about choosing one family and more about deciding which trade-offs to prioritize. The following chapters examine how these trade-offs manifest in the supervision strategies and qualitative outputs of the selected models.

5.6 Representation and Editability

Editability is one of the defining advantages of vector graphics and a central requirement in real design workflows. Unlike raster images, SVG graphics are expected to support direct manipulation of individual elements, including moving shapes, adjusting colors, reorganizing layers, or modifying geometry without affecting unrelated parts of the graphic. Representation plays a central role in determining how editable a generated SVG is.

Parametric representations offer the most direct path to editability because each element corresponds to explicit geometric parameters. However, this advantage depends heavily on path count and geometric organization. When models such as LIVE or SAMVG introduce large numbers of paths to match visual detail, the output becomes theoretically editable but practically difficult to work with. Sequential representations such as those used in DeepSVG and DeepIcon produce procedurally structured output, but editing a single visual element requires locating its corresponding command segment, which is not always straightforward. Implicit representations shift geometry into neural parameter space, meaning individual shapes are not directly accessible until the network output is converted into discrete primitives. How clean this conversion is determines whether the final SVG supports practical editing.

An important observation is that editability is not determined by representation type alone. Initialization strategy, supervision signals, and structural priors all contribute. Methods that incorporate segmentation, attention-based masking, or layer-aware initialization tend to produce outputs with clearer object boundaries regardless of their underlying representation. LayerTracer is a clear example: by training on human construction sequences, it learns to produce layer-separated outputs as a direct consequence of its training data rather than through explicit structural constraints.

This suggests that editability emerges from the interaction between representation, optimization strategy, and structural design rather than from representation alone. Representations that maintain explicit primitives support local editing more naturally, while representations that emphasize abstraction improve generative flexibility at the cost of direct control. Hybrid approaches attempt to balance this by preserving layered structure while allowing flexible optimization or semantic guidance.

5.7 Structural Properties Emerging from Representation

Beyond editability, representation choices directly influence the structural properties of generated SVG graphics. Structural quality refers to how well a vector graphic preserves

meaningful organization, including hierarchy, grouping, layering, and geometric consistency. While visual similarity can be achieved through many small primitives, structural properties determine whether the SVG remains interpretable and aligned with design practices.

Parametric optimization-based approaches frequently illustrate the tension between visual accuracy and structural quality. Because these methods refine primitives to minimize image-space loss, they tend to introduce redundant paths or fragmented geometry to capture fine visual detail. This improves reconstruction quality but can result in overlapping shapes and inconsistent grouping. Sequential representations produce structure implicitly through command order rather than explicit hierarchy, which can leave structural relationships ambiguous even when command prediction is accurate. Implicit neural representations allow geometry to emerge from continuous neural functions, enabling smoother global organization and layered outputs, but structural clarity depends on how carefully the conversion into discrete SVG elements is handled. Latent representations capture structural regularities from training data, which can lead to more coherent outputs compared to purely optimization-based methods, but clean hierarchy is not guaranteed unless structural signals are explicitly modeled during training.

A key observation is that structural properties in most models emerge indirectly. The majority of approaches optimize visual objectives, with structure appearing as a byproduct rather than a primary optimization target. This explains why many methods achieve strong visual fidelity but still produce SVGs with redundant primitives or weak hierarchy. Initialization strategies partially address this: methods that begin from segmentation masks, attention priors, or layer-aware starting points tend to produce cleaner organization because they reduce the need for excessive path refinement. Representation defines the space in which structural organization can appear, but initialization and supervision determine whether that structure remains coherent in the final output.

5.8 Summary

This chapter examined representation as a central analytical dimension for understanding SVG generation methods. Although all models ultimately produce SVG files, the way vector graphics are encoded during generation introduces fundamental differences in structure, editability, and visual behavior. Representation therefore acts as a unifying perspective for interpreting the design decisions underlying different approaches.

Four primary representation families were identified: parametric, sequential, implicit neural, and latent. Parametric representations provide direct geometric control and strong interpretability but may produce fragmented structures under complex optimization. Sequential representations capture procedural drawing behavior but struggle with

scalability and explicit hierarchy. Implicit neural representations enable flexible topology and expressive layered outputs while shifting optimization into neural parameter space. Latent representations offer scalable generative modeling and global style control at the cost of fine-grained geometric control.

Across these families, representations that maintain explicit primitives tend to support local editing and structural transparency, whereas representations that emphasize abstraction improve generative flexibility. Hybrid approaches combine explicit structure with learned or semantic guidance to navigate this trade-off. Editability and structural quality rarely emerge from representation alone but depend on the interaction between representation, initialization strategy, and supervision signals.

These observations provide the foundation for the next analytical dimension of the thesis. While representation defines how vector graphics are encoded, supervision determines how models learn to shape that representation. The following chapter examines supervision signals and guidance strategies, analyzing how they interact with representation choices to influence the quality, structure, and behavior of generated SVG graphics.

6 Supervision and Guidance Strategies for SVG Generation

While representation defines how vector graphics are encoded during generation, supervision determines how those representations are shaped. In SVG generation, supervision refers to the signals that guide learning or optimization so that produced vector graphics align with a target objective. These signals may come from paired datasets, reconstruction losses, semantic guidance models, or structural priors introduced during initialization.

An important distinction is that supervision in this domain extends beyond dataset labels. Many methods do not rely on ground-truth SVG targets at all. Instead, they optimize vector representations using raster reconstruction, semantic similarity, or pre-trained generative models as guidance. Supervision is therefore a core architectural decision: it determines which aspects of the graphic are emphasized during generation and shapes how structure emerges.

Supervision also interacts closely with representation. Because gradients propagate through the chosen representation, the same supervision signal can produce different geometric outcomes depending on how the vector graphic is encoded. Image-based losses applied to explicit primitives may introduce additional paths to match appearance, whereas the same losses applied to implicit neural representations may reshape continuous fields. This interaction highlights that supervision and representation need to be analyzed together.

A further distinction concerns when supervision is applied. Some models learn reusable generation behavior from datasets at training time. Others apply supervision directly during inference, refining a vector representation for each individual input. This difference has significant consequences for speed, structure, reproducibility, and practical usability, and is examined in detail in Section 6.4.

The following sections examine the main supervision families used across the selected models, focusing on how different guidance strategies influence structure, editability, visual quality, and practical usability.

6.1 Reconstruction-Based Supervision

Reconstruction-based supervision guides generation by comparing a rendered SVG output with a target raster image. The objective is to minimize the difference between prediction and reference using loss functions defined in image or feature space. Common choices include pixel-wise L1 or L2 losses and perceptual losses computed from pretrained convolutional networks [9]. This paradigm is particularly common in dataset-trained models and image-to-SVG approaches.

One advantage of reconstruction-based supervision is stability. Because the target is known, gradients provide direct information about how geometry should change. Training behavior is predictable, and when datasets contain clean SVG examples, models can learn typical path counts, grouping patterns, and structural conventions. The dataset itself acts as the primary supervision signal: models such as DeepSVG [4], DeepIcon [32], and Im2Vec [29] are trained to reconstruct vector graphics from data, and the reconstruction objective is what drives learning. As discussed in Chapter 4, these models do not apply supervision at inference time, once trained, generation occurs through a forward pass.

However, reconstruction-based supervision has limits. Output quality depends heavily on dataset distribution: models trained on simplified icons may generalize poorly to complex scenes, and datasets with inconsistent SVG structure can transfer those inconsistencies to generated outputs. More importantly, reconstruction losses measure visual similarity rather than structural correctness. A model can achieve low reconstruction error while producing redundant paths or weak hierarchy, particularly when pixel-level matching dominates the objective. Without explicit structural constraints, reconstruction supervision does not guarantee editability or geometric clarity.

6.2 Semantic Guidance with Vision–Language Models

Many SVG generation tasks operate without paired SVG references. In text-to-SVG settings, the only available signal may be a textual description of the desired graphic. Semantic guidance addresses this by using pretrained vision–language models as evaluators that provide feedback about whether the rendered SVG aligns with a prompt.

As described in Chapter 3, CLIP embeds images and text into a shared representation space [12]. In SVG generation pipelines, the rendered vector graphic is passed through the image encoder and compared against the encoded text prompt. Because gradients can propagate back through the differentiable rasterizer, vector parameters can be adjusted to improve semantic alignment. This enables open-ended generation without

requiring paired SVG datasets.

Semantic guidance offers flexibility across diverse prompts and captures high-level visual semantics that are difficult to encode through handcrafted losses. At the same time, it remains indirect: the signal measures alignment in embedding space rather than geometric correctness. Optimization may therefore introduce additional primitives or unstable geometry to satisfy semantic objectives, without any guarantee of clean structure or editability.

In practice, semantic guidance is rarely used in isolation. It typically acts as an intermediate step or is combined with diffusion-based supervision, as in VectorFusion [16] and early SVGDreamer configurations. This reflects a broader pattern: CLIP guidance is most useful for conceptual alignment, but stronger perceptual signals are usually needed to achieve visually convincing results.

6.3 Diffusion-Based Supervision

Diffusion-based supervision leverages pretrained diffusion models as perceptual priors rather than relying on explicit targets or embedding similarity alone. These models capture large-scale image distributions and provide gradients that encourage rendered SVGs to move toward visually plausible solutions conditioned on a text prompt.

The most common implementation is Score Distillation Sampling (SDS), introduced by DreamFusion [15]. SDS removes the need for ground-truth images by treating a frozen diffusion model as the optimization signal — it evaluates the rendered SVG and provides gradients that push vector parameters toward outputs the diffusion model considers plausible for the given prompt. Variational Score Distillation (VSD), used in SVGDreamer [17], refines this further by reducing the oversaturation artifacts that plain SDS tends to produce.

Several models in this thesis use diffusion-based supervision as their primary guidance signal, including VectorFusion, SVGDreamer [17], SVGDreamer++ [30], NeuralSVG [31], NIVeL [5], and T2V-NPR [34].

The main advantage is expressive power. Because diffusion models encode rich visual knowledge, they can guide generation toward complex compositions and stylistic diversity that reconstruction losses alone cannot achieve. However, diffusion gradients operate in pixel space and primarily emphasize perceptual realism rather than geometric simplicity. Without additional structural constraints, optimization tends to increase primitive count to satisfy visual objectives, which directly affects editability. Inference cost is also substantially higher than for forward-decoding approaches, since each generation

requires repeated rendering and evaluation cycles.

Despite these trade-offs, diffusion-based supervision has become the dominant strategy for open-ended SVG generation because it removes the dependency on paired datasets while providing strong perceptual guidance. Most recent models combine it with representation innovations or structural priors to mitigate geometric instability, which is why supervision cannot be analyzed independently from the other design choices examined in this thesis.

6.4 Dataset Supervision vs. Per-Instance Supervision

A fundamental distinction across SVG generation methods is when supervision is applied. Dataset-supervised approaches learn generation behavior from collections of SVG graphics during training. Per-instance supervision applies guidance during generation itself, refining a vector representation for each individual input or prompt.

For dataset-supervised models, the dataset is the supervision signal. Models such as DeepSVG [4], DeepIcon [32], and Im2Vec [29] are trained on reconstruction objectives against SVG data, and once trained, generation requires no further optimization. LayerTracer [33] takes a specific variant of this approach: it is trained on a synthetically generated serpentine dataset, where vector graphics are constructed in a deliberate layer order that mimics human drawing behavior [33]. The dataset structure itself encodes the supervision prior, teaching the model to generate layered, sequential outputs that reflect design conventions rather than arbitrary path orderings.

Per-instance supervision works differently. Models such as LIVE [3], SAMVG [27], VectorFusion [16], SVGDreamer [liSVGDreamerTexttoSVG2023], NeuralSVG [liNeuralSVGTexttoS], NIVeL [zhaoNIVeLTexttoVector2023], and T2V-NPR [liT2V-NPRTexttoVector2023] apply supervision directly during generation. No reusable generation model is trained; instead, vector parameters are optimized from scratch for each input using reconstruction losses, semantic alignment, or diffusion priors. This is computationally expensive but allows structure to emerge dynamically in response to each specific prompt or image.

Some models combine both paradigms. SVGFusion [35] learns latent representations from SVG datasets during training, then applies optimization-based refinement at inference time. This hybrid approach attempts to balance the structural consistency of dataset supervision with the flexibility of per-instance optimization. The same pattern appears in T2V-NPR [34], which learns path-level representations from data before using diffusion guidance during generation.

This distinction has direct practical implications. Dataset-supervised models produce

consistent outputs and support fast inference, but their structure is constrained by what the training data contains. Per-instance supervision offers greater adaptability but introduces variability due to initialization choices and stochastic guidance, and makes systematic evaluation across methods more difficult. Hybrid approaches mitigate some of these trade-offs but increase overall system complexity.

6.5 Structural Priors and Initialization as Supervision

Beyond explicit loss functions, many SVG generation methods rely on structural priors and initialization strategies that act as additional forms of supervision. These mechanisms are not always described as supervision, but they strongly influence how structure emerges in the resulting SVG.

Initialization determines the starting point of optimization. Random initialization provides flexibility but often leads to unstable optimization and fragmented paths. Structured initialization introduces prior knowledge about object boundaries and layout, reducing the need for excessive refinement. SAMVG [27] illustrates this by using segmentation outputs to define an initial set of semantically meaningful regions before vector optimization begins. LIVE [3] and SVGDreamer++ [liSVGDreamerTexttoSVG2023] use layered initialization strategies that progressively refine structure across stages. In these cases, the initialization itself constrains geometry before any loss is applied.

Structural priors can also come from learned representations. SVGFusion [35] and T2V-NPR [liT2V-NPRTexttoVector2023] both use latent starting points shaped by prior training, so optimization begins from a representation that already encodes typical path structures rather than from scratch. This often improves convergence speed and reduces structural noise compared to random initialization.

The interaction with diffusion supervision is particularly important here. Because diffusion gradients operate in pixel space and do not enforce hierarchy, initialization plays a critical role in determining whether structure remains interpretable. When meaningful priors are absent, diffusion-driven optimization tends to increase primitive count to match visual detail. When structured initialization is present, the same supervision signal can produce more coherent geometry with fewer paths. This is one reason why models with similar supervision strategies can produce very different structural outcomes depending on how generation is initialized.

From a broader perspective, treating initialization and structural constraints as forms of supervision helps explain observations that loss functions alone cannot account for. Methods that incorporate segmentation, layering, or learned latent starting points often generate more interpretable SVG outputs despite relying on similar perceptual objectives

as less structured alternatives.

6.6 Summary

This chapter analyzed supervision as a key dimension shaping SVG generation methods. While representation determines how vector graphics are encoded, supervision determines how these representations evolve toward desired outputs.

Reconstruction-based supervision provides explicit targets and stable optimization, but its quality depends on dataset distribution and it does not guarantee structural correctness. Semantic guidance enables generation without paired data, but its indirect nature can lead to unstable geometry. Diffusion-based supervision provides strong perceptual priors that improve visual expressiveness significantly, at the cost of increased computational load and structural complexity.

The distinction between dataset supervision and per-instance supervision clarifies how methods allocate computational effort. Dataset-trained models use the training data itself as the supervision signal and favor efficiency and consistency. Per-instance optimization offers flexibility and expressive power at the cost of runtime and reproducibility. Hybrid approaches such as SVGFusion [35] and T2V-NPR [**liT2V-NPRTexttoVector2023**] combine both, reflecting an ongoing attempt to balance speed with adaptability. LayerTracer [3] represents a particularly direct example of how dataset design can encode structural priors: its serpentine training data teaches the model to produce layered outputs without requiring any explicit structural loss.

Structural priors and initialization strategies extend the definition of supervision beyond explicit loss functions. They shape the space in which optimization occurs and strongly influence geometry quality, often more than the choice of loss function itself.

Across all supervision strategies, a recurring pattern emerges: supervision tends to prioritize visual objectives, while structural properties emerge indirectly. This explains why models with strong perceptual guidance can produce visually convincing SVGs that are difficult to edit. When analyzed together with representation, supervision provides a framework for understanding why different model architectures produce distinct structural outcomes. The next chapter examines how these properties can be measured and compared through evaluation.

7 Evaluation

Evaluating SVG generation methods is fundamentally different from evaluating raster image generation. While visual realism remains an important criterion, vector graphics introduce additional dimensions related to structure, editability, and usability. As a result, evaluation in SVG generation cannot rely solely on traditional image-based metrics and must instead be understood as a multi-dimensional design problem.

In raster image generation, evaluation typically focuses on perceptual similarity, distribution alignment, or semantic correctness. Metrics such as FID, CLIP score, or perceptual similarity are widely used to quantify visual quality. However, SVG graphics are not only visual artifacts but also structured representations composed of geometric primitives, hierarchical relationships, and editable components. This means that two SVGs with similar visual appearance may differ significantly in their internal structure and practical usability.

This distinction creates a gap between how SVG generation methods are commonly evaluated and what makes generated vector graphics useful in practice. Many works emphasize visual fidelity because it is easier to measure quantitatively, while structural aspects such as hierarchy, redundancy, or editability are often assessed qualitatively through visual inspection. Consequently, evaluation practices may overlook properties that are critical for downstream design workflows.

Another challenge arises from the diversity of generation paradigms. Dataset-trained models, optimization-based approaches, and hybrid pipelines produce outputs with different characteristics. Some methods prioritize compact structure, while others favor visual realism even if this leads to complex geometry. Without clearly defined evaluation dimensions, comparisons across models become inconsistent and may depend on selective qualitative examples or task-specific metrics.

Evaluation is therefore not a neutral reporting step but a design decision that reflects what researchers consider important. Choosing metrics implicitly prioritizes certain properties of generated SVGs over others. Metrics focused on appearance reward perceptual realism, whereas metrics that capture geometric or structural properties would emphasize editability and interpretability. The absence of standardized structural metrics explains why visually impressive results may still produce SVGs that are difficult to modify.

This issue is closely related to the broader research gap identified in this thesis. Although a growing number of SVG generation methods have been proposed, systematic evaluation frameworks that consider both visual and structural dimensions remain limited. Existing benchmarks primarily adopt image-generation metrics, which do not fully reflect the characteristics that distinguish vector graphics from raster outputs.

From this perspective, evaluation should be treated as a design problem rather than a purely technical procedure. Defining evaluation criteria requires clarifying the intended use of generated SVGs, whether for illustration, icon design, vectorization, or interactive editing. Different applications may prioritize different properties, suggesting that evaluation must account for multiple dimensions rather than a single aggregate score.

For these reasons, this chapter examines evaluation practices in SVG generation through several complementary dimensions. The following sections analyze visual quality metrics, structural quality indicators, editability considerations, and practical constraints such as computational cost. Together, these perspectives form a foundation for a multi-dimensional evaluation framework that better reflects the goals and trade-offs of modern SVG generation methods.

7.1 Evaluation of Visual Quality

Visual quality remains the most commonly reported evaluation dimension in SVG generation research. Because generated vector graphics are ultimately rendered as images, many works adopt metrics originally developed for raster image generation to quantify realism, semantic alignment, and perceptual similarity. This practice provides a convenient way to compare methods but also introduces important limitations when applied to structured vector outputs.

A widely used metric is the Fréchet Inception Distance (FID), which measures the similarity between distributions of generated and real images in a learned feature space. In SVG generation, outputs are rasterized before computing FID, allowing comparison with raster datasets or rendered SVG references. Lower FID scores indicate that generated samples resemble the target distribution more closely. As a result, several recent works report FID as evidence of improved visual realism.

However, FID primarily evaluates appearance rather than structure. Two SVGs that render to visually similar images may have very different geometric complexity, path counts, or hierarchical organization. Consequently, improvements in FID do not necessarily correspond to improvements in editability or structural clarity. This limitation is particularly relevant for optimization-based methods, which may achieve strong perceptual alignment by introducing many primitives.

Another commonly used evaluation signal is CLIP-based similarity. CLIP score measures the semantic alignment between rendered SVG images and textual prompts, making it useful for text-to-SVG tasks. Because CLIP captures high-level semantic relationships, it complements FID by evaluating conceptual correctness rather than distribution similarity. Nevertheless, like FID, CLIP operates in image space and does not directly reflect geometric properties of the vector representation.

Perceptual similarity metrics, such as feature-based distances computed from pretrained networks, are also used to assess how closely generated SVGs match reference images. These metrics are particularly relevant for vectorization tasks, where reconstruction accuracy is important. While perceptual losses can encourage better visual fidelity during training, their use as evaluation metrics shares the same limitation: they measure rendered appearance rather than structural quality.

Human evaluation remains an important complementary approach. Qualitative comparisons, user studies, or curated visual examples are frequently used to demonstrate improvements that are difficult to capture numerically. Researchers often highlight representative samples that illustrate stylistic diversity, realism, or prompt alignment. Although this provides valuable insight, qualitative evaluation introduces subjectivity and may emphasize best-case examples rather than typical outputs.

This leads to an important observation about evaluation practices in the literature. Visual metrics are often reported selectively, and different works adopt different combinations of metrics depending on the task. In some cases, quantitative results are presented alongside carefully chosen qualitative examples that highlight strengths while structural limitations remain underreported. This does not imply incorrect evaluation, but it indicates that current practices emphasize visual performance more strongly than structural characteristics.

Despite these limitations, visual quality evaluation remains essential. It provides a baseline for assessing realism, semantic correctness, and reconstruction performance, all of which are important aspects of SVG generation. However, visual metrics alone cannot fully characterize the usefulness of generated vector graphics. Recognizing this limitation motivates the need to examine additional evaluation dimensions that capture properties unique to vector representations, beginning with structural quality.

7.2 Evaluation of Structural Quality

While visual metrics assess how SVG outputs appear after rasterization, structural quality evaluates the internal organization of the vector graphic itself. This includes properties such as path redundancy, hierarchy, layering, grouping, and topology stability.

Because SVG is a structured representation, these properties directly influence whether the generated output is interpretable, reusable, and suitable for downstream editing.

Structural quality is rarely measured through standardized metrics. Instead, many works report structural observations qualitatively, for example by visualizing path counts or showing layered decompositions. Some papers discuss compactness by reporting the number of primitives used to represent a graphic, implicitly assuming that fewer paths correspond to cleaner structure. Although path count provides a useful signal, it does not fully capture hierarchy or grouping relationships between elements.

Redundancy is one of the most common structural issues observed in optimization-based methods. When supervision emphasizes pixel-level accuracy, models may introduce overlapping or fragmented paths in order to match appearance. This can lead to vector graphics that render correctly but contain unnecessary geometric complexity. Structural evaluation therefore often involves inspecting whether primitives correspond to meaningful visual components rather than small corrective adjustments.

Hierarchy and layering represent another important dimension of structural quality. Well-structured SVG graphics organize elements into groups and layers that reflect semantic relationships. Some recent methods explicitly model layered representations or hierarchical optimization strategies, which can improve structural clarity. However, the degree to which hierarchy is preserved varies widely across models and is rarely quantified in a consistent manner.

Topology stability is also relevant when analyzing structural quality. Small changes in prompts or initialization may cause significant variations in primitive arrangement, even when visual appearance remains similar. This instability can make it difficult to interpret the generated structure or reuse it across related tasks. Evaluating topology stability therefore involves examining whether generated SVGs maintain consistent organization across variations.

Dataset-trained models introduce a different structural behavior. Because they learn from examples, they may inherit typical path structures and grouping patterns present in the dataset. This can lead to more consistent geometry, especially for icon-style graphics. However, dataset bias may also restrict structural diversity or limit generalization to more complex scenes.

Another challenge in structural evaluation is the absence of unified benchmarks. Unlike visual metrics, which can be computed automatically after rasterization, structural properties require parsing SVG files and defining criteria that capture meaningful organization. This makes evaluation more context-dependent and explains why structural quality is often discussed descriptively rather than measured quantitatively.

Despite these challenges, structural evaluation is essential for understanding the prac-

tical value of generated SVGs. Graphics with clear hierarchy, limited redundancy, and stable topology are more likely to support editing and reuse. Conversely, structurally complex outputs may require manual cleanup before they can be integrated into design workflows.

Recognizing structural quality as a distinct evaluation dimension highlights a key limitation of current evaluation practices. Visual similarity does not guarantee structural clarity, and improvements in perceptual metrics may coincide with increased geometric complexity. This motivates the need to consider editability as an explicit evaluation dimension, which is examined in the next section.

7.3 Editability

Editability is one of the defining characteristics that differentiates vector graphics from raster images, yet it is rarely formalized as an evaluation criterion in SVG generation research. In practice, the usefulness of a generated SVG is strongly determined by how easily a designer can modify it after generation. This includes the ability to move elements independently, change colors, adjust shapes, reorganize layers, and reuse components without extensive manual cleanup.

Unlike visual quality, editability cannot be captured by a single numerical metric. Instead, it emerges from several structural properties, including meaningful grouping, limited path redundancy, stable topology, and clear separation between visual components. An SVG that renders correctly but contains hundreds of overlapping primitives may be visually convincing while remaining difficult to edit. Conversely, a slightly less detailed graphic with clean structure may be significantly more useful in practice.

Several observable indicators can be used to assess editability. These include whether primitives correspond to semantic parts of the graphic, whether layers reflect meaningful organization, whether path counts remain manageable, and whether local edits affect only intended regions. Methods that explicitly model layers, incorporate structural priors, or learn from clean SVG datasets often produce outputs that score higher along these dimensions.

Representation and supervision jointly influence editability outcomes. Parametric representations with explicit primitives generally support direct manipulation, but optimization-driven supervision may introduce redundant paths that reduce clarity. Implicit neural representations can produce coherent layered outputs, yet the conversion process into discrete primitives determines whether editing remains feasible. Dataset-trained approaches may inherit structured patterns from data, though they can also propagate dataset-specific limitations.

Another important aspect of editability is predictability. Designers benefit from consistent structure across similar generations, enabling reuse and incremental refinement. Methods that produce highly variable primitive arrangements may hinder this workflow, even when individual outputs appear visually strong. This highlights that editability is closely linked to both structural stability and reproducibility.

In real design environments, editability is often evaluated informally by importing generated SVGs into tools such as Figma or Illustrator and inspecting their structure. Designers assess whether elements can be selected individually, whether layers are organized logically, and whether modifications introduce unexpected visual artifacts. Although this evaluation is qualitative, it reflects practical requirements that are not captured by visual metrics alone.

The lack of standardized editability metrics represents a notable gap in current research. While some works discuss compactness or layering qualitatively, few provide systematic criteria for assessing usability from a design perspective. This suggests that editability should be treated as a first-class evaluation dimension rather than an implicit byproduct of structural quality.

Considering editability explicitly allows evaluation to better reflect the intended use of generated SVGs. For applications such as icon design, interface assets, and vector illustration, the ability to modify and reuse generated graphics is often more important than maximizing perceptual realism. Recognizing this distinction helps explain why different methods may be preferred depending on the application context.

The next section extends the evaluation perspective beyond visual and structural properties by examining efficiency and practical constraints, which influence whether SVG generation methods can be deployed in interactive or resource-constrained settings.

7.4 Efficiency and Practical Usability

Beyond visual quality, representation, and structural properties, practical usability of SVG generation methods is strongly influenced by efficiency. In many real design scenarios, generation speed, computational cost, and hardware requirements determine whether a method can be integrated into interactive workflows or remain primarily research-oriented.

SVG generation approaches differ significantly in inference behavior. Optimization-based methods typically require iterative refinement, where vector parameters are repeatedly updated using gradient signals from reconstruction or semantic guidance. This process can produce high-quality and flexible results, but it introduces latency. Gener-

ating a single SVG may take several minutes depending on the number of primitives, optimization steps, and guidance models involved. As a result, these methods are often less suitable for real-time interaction unless simplified or accelerated.

Neural implicit approaches share similar efficiency challenges. Because geometry is encoded in neural network weights, generation involves iterative optimization in parameter space combined with repeated rendering. This increases computational cost compared to forward decoding models. While implicit representations can produce coherent layered structures, the need for continuous evaluation of neural fields and rasterization can limit scalability in practical applications.

Dataset-trained feed-forward models generally offer faster inference. Once trained, these models generate SVG outputs in a single forward pass, making them more compatible with interactive tools. This property explains why sequential and latent-based methods are attractive for deployment scenarios, even when they may provide less control over structure compared to optimization-based approaches.

However, efficiency is not determined solely by inference time. Hardware requirements also play an important role. Many recent methods rely on large pretrained models, including diffusion systems and multimodal encoders, which introduce significant memory and compute demands. High-VRAM GPUs may be required even when the SVG output itself is relatively simple. This creates a gap between conceptual feasibility and practical reproducibility.

Another dimension of efficiency relates to scalability with graphic complexity. Methods that maintain a fixed representation size, such as latent models, may scale more gracefully as scene complexity increases. In contrast, approaches that directly optimize primitives may experience rapidly increasing computation as path counts grow. This trade-off reflects a broader tension between structural control and computational cost.

Efficiency also interacts with editability. Iterative optimization allows fine-grained control but slows down generation, while fast feed-forward models may sacrifice structural refinement for speed. In practice, different application contexts prioritize different points along this spectrum. Interactive design tools require responsiveness, whereas offline generation pipelines may tolerate longer optimization.

Reproducibility is another practical constraint highlighted across many SVG generation works. Public implementations, pretrained weights, and clear documentation significantly affect whether methods can be evaluated or adopted by other researchers and practitioners. The absence of reproducible pipelines limits comparative analysis and contributes to fragmented evaluation practices within the field.

Considering efficiency alongside representation and structure provides a more complete

understanding of SVG generation methods. Rather than treating computational cost as a secondary concern, it becomes part of the evaluation framework that explains why certain architectures are favored in applied settings.

With representation, structure, editability, and efficiency discussed as complementary dimensions, the next chapter synthesizes these observations to analyze how architectural choices collectively influence SVG generation outcomes and to identify patterns that emerge across the selected models.

7.5 Quantitative vs Qualitative Evaluation Gap

Evaluation plays a central role in generative modeling, as it determines how progress is measured and how different methods are compared. In SVG generation research, evaluation typically combines quantitative metrics with qualitative visual analysis. However, a consistent mismatch exists between what is measured numerically and what matters for vector graphics in practice. This mismatch forms an important gap that directly motivates the analysis in this thesis.

Most SVG generation papers report quantitative metrics adapted from raster image generation. Common examples include Fréchet Inception Distance (FID), reconstruction losses, perceptual similarity scores, and occasionally CLIP-based alignment metrics. These metrics evaluate how visually similar rendered SVG outputs are to reference images or how well generated images match textual prompts. Because SVGs are rendered into raster images before evaluation, the measurement process primarily reflects pixel-level appearance rather than vector structure.

While such metrics provide useful signals about realism and semantic alignment, they do not capture key properties that define vector graphics quality. SVG outputs may achieve strong FID scores while containing redundant paths, fragmented geometry, unstable layering, or poor grouping. From a design perspective, these issues significantly affect editability and reuse, yet they remain largely invisible to standard quantitative metrics.

Another limitation arises from how quantitative results are reported. Many papers highlight best-case metrics obtained under specific configurations, datasets, or prompt selections. This does not necessarily reflect typical performance or structural robustness across diverse scenarios. As a result, quantitative evaluation can sometimes function more as a comparative indicator within a narrow experimental setup than as a reliable measure of practical SVG quality.

Qualitative evaluation, on the other hand, focuses on visual inspection and structural

interpretation. Papers often include visual examples that illustrate layering behavior, path simplicity, or stylistic consistency. For vector graphics, these qualitative aspects are essential because they reveal whether the generated SVG behaves as an editable artifact rather than merely a rendered image.

However, qualitative evaluation introduces its own challenges. Visual inspection is inherently subjective and difficult to standardize. Different works emphasize different aspects, such as realism, style diversity, icon clarity, or structural cleanliness. Without shared criteria, comparisons across methods become difficult, which contributes to fragmented understanding of model behavior.

This gap between quantitative metrics and qualitative assessment is particularly important when considering the research questions of this thesis. The goal is not only to evaluate visual realism but to understand how representation and supervision choices influence structural outcomes. This requires evaluation perspectives that go beyond image similarity and explicitly consider properties such as path organization, hierarchy stability, editability, and computational cost.

Several emerging works implicitly acknowledge this issue by introducing task-specific analyses, such as layer consistency studies, path simplification statistics, or user-centric editing demonstrations. Nevertheless, these approaches remain inconsistent and are rarely adopted as standard benchmarks.

Therefore, the evaluation dimension in SVG generation should be understood as multi-layered. Quantitative metrics capture visual fidelity and semantic alignment, while qualitative analysis reveals structural correctness and usability. Neither perspective alone is sufficient.

Recognizing this evaluation gap provides a conceptual bridge to the final synthesis chapter. By analyzing representation, supervision, structure, editability, efficiency, and evaluation together, it becomes possible to identify patterns that explain why certain architectural combinations produce SVG outputs that are not only visually convincing but also structurally meaningful.

7.6 Toward a Multi-Dimensional Framework for Evaluating SVG Quality

The analysis throughout this thesis suggests that evaluating SVG generation cannot rely on a single metric. Unlike raster images, SVG outputs must be interpreted not only visually but also structurally and functionally. In practice, a high-quality SVG is expected

to be semantically correct, geometrically stable, structurally clean, and editable.

Existing literature typically evaluates models using visual similarity metrics or qualitative examples. However, these signals do not fully capture the properties that determine whether an SVG is useful in real design workflows. Based on the reviewed methods and their outputs, this thesis proposes a multi-dimensional perspective describing what aspects of SVG quality should be measured.

Semantic Alignment A first dimension concerns whether the generated SVG matches the intended concept. In text-to-SVG settings, this corresponds to semantic alignment between the prompt and the rendered output. Most works approximate this using image-text similarity measures derived from pretrained models (e.g., CLIP) or diffusion-based guidance signals.

From an evaluation perspective, semantic quality can therefore be approximated using a small number of alignment metrics, such as CLIP similarity or prompt-image matching scores. These metrics indicate whether the generated graphic represents the correct object or concept, but they do not reflect how the SVG is constructed internally.

Structural Cleanliness A second dimension relates to the internal organization of the SVG. Across optimization-based methods, a recurring issue is the emergence of redundant paths, fragmented shapes, or unstable geometry. Even visually convincing outputs may contain excessive primitives that reduce interpretability.

Structural cleanliness can be approximated through measurable signals such as number of paths, presence of redundant primitives, layer consistency, and grouping coherence. Models that explicitly incorporate layered representations, such as NeuralSVG or Layer-Tracer, tend to produce outputs where structure more closely reflects design logic. This suggests that structural organization is not only an aesthetic property but an indicator of representation quality.

Editability Editability describes whether a generated SVG can be realistically modified by a designer. This property is rarely quantified but frequently illustrated qualitatively in model papers. In practice, editability is influenced by path decomposition, layering, parameter stability, and semantic grouping.

Potential indicators of editability include primitive count relative to visual complexity, layer separability, stability of shapes under small parameter changes, and whether objects correspond to meaningful components. For example, results showing complex scenes

constructed from a limited number of shapes (as demonstrated in NeuralSVG) suggest higher editability compared to outputs requiring hundreds of primitives.

Geometric Quality Another important dimension concerns geometric fidelity. Optimization-based methods may produce jagged curves, unstable boundaries, or small artifacts that are visually subtle but structurally problematic. These issues affect downstream editing and reuse.

Geometric quality can be approximated by examining curve smoothness, continuity of paths, presence of self-intersections, and boundary accuracy relative to the rendered target. While many papers show qualitative comparisons, explicit geometric metrics remain underexplored.

Complexity and Efficiency Finally, SVG quality is related to representational efficiency. Two outputs with similar visual appearance may differ substantially in complexity, generation time, and computational requirements. Iterative optimization methods often trade efficiency for flexibility, whereas dataset-trained models shift complexity toward training.

Indicators in this dimension include inference time, number of primitives, memory footprint, and reproducibility requirements. These signals are especially relevant when considering real design workflows where generation speed and controllability matter.

Summary Taken together, these observations suggest that SVG quality should be evaluated across multiple complementary dimensions rather than a single score. Semantic alignment reflects whether the correct concept is generated, structural cleanliness captures representation organization, editability reflects practical usability, geometric quality describes shape stability, and efficiency characterizes computational practicality.

This framework does not introduce new quantitative metrics but instead clarifies what should be measured when comparing SVG generation methods. It provides a structured lens for interpreting results across different architectural choices and serves as a foundation for the cross-model synthesis presented in the next chapter.

8 Comparative synthesis

8.1 Structure Patterns

One of the most consistent observations across the analyzed models is that the internal representation of vector graphics strongly influences the structural properties of the resulting SVG. While different approaches aim to generate visually convincing graphics, the way vector content is represented inside the model often determines whether the output is fragmented, layered, editable, or difficult to interpret.

Parametric representations, commonly used in optimization-based methods, directly operate on explicit SVG primitives such as Bézier control points, colors, and stroke parameters. This representation provides clear control over geometry and allows gradients to update shapes directly. However, the analysis shows that this directness can also lead to structural instability. When supervision is applied primarily in raster space, models may increase the number of primitives to minimize visual error, resulting in redundant paths and fragmented structure. Methods such as LIVE or diffusion-guided optimization approaches frequently exhibit this behavior, where visual fidelity improves at the cost of structural simplicity.

Sequential representations introduce a different structural pattern. By modeling SVG graphics as ordered command sequences, these approaches capture procedural drawing logic rather than static geometry. Models like DeepSVG demonstrate that hierarchical sequence modeling can produce coherent icon-level structure, especially in constrained datasets. However, sequential representations often struggle when scenes become complex, as long command chains amplify error propagation and make global structural consistency difficult to maintain. As a result, sequence-based methods tend to produce clean structure in simple settings but face scalability limitations.

Implicit representations shift the representation from explicit primitives to neural parameter space. Instead of directly optimizing paths, models such as NeuralSVG or NIVeL learn neural functions that define shapes or layered compositions. This representation introduces an important structural effect: layering emerges as part of the representation itself rather than as an optimization artifact. Because the number of layers is often constrained, implicit approaches can produce SVGs with fewer primitives and clearer

decomposition. The analysis suggests that this constraint acts as an implicit structural regularizer, encouraging simpler and more interpretable outputs. At the same time, the mapping between neural parameters and explicit SVG primitives becomes less direct, which may complicate fine-grained manual editing.

Latent representations introduce another structural behavior by separating generation from explicit geometry construction. In models such as SVGFusion or T2V-NPR, vector graphics are generated through latent embeddings that encode structural patterns learned from datasets. This allows the model to inherit dataset-level regularities, often producing more coherent path organization compared to purely optimization-driven methods. However, because structure is mediated through a learned latent space, direct control over primitive composition may be reduced. This creates a trade-off between structural coherence and explicit controllability.

Across these representation families, a recurring pattern emerges: representations that constrain structure tend to produce cleaner SVGs, while representations that maximize flexibility often increase structural complexity. Explicit parametric optimization provides strong geometric control but risks path redundancy. Sequential approaches capture procedural structure but face scalability challenges. Implicit and layered neural representations encourage compact structure through architectural constraints. Latent representations inherit structural priors from data but reduce direct interpretability.

Importantly, these observations suggest that representation choice does not merely affect how SVGs are generated, but also what kinds of structures are possible. In other words, structural quality is not only an optimization outcome but a representational property. This insight provides a foundation for understanding why certain models produce outputs that appear more editable or design-like, even when visual fidelity across methods is comparable.

The next section builds on this perspective by examining how supervision signals interact with representation choices, further shaping geometric behavior and structural outcomes.

8.2 Supervision → Geometry Behavior

While representation determines what structural forms are possible, supervision determines which of those forms are encouraged during generation. Across the reviewed literature, supervision signals play a central role in shaping geometry, influencing not only visual similarity but also primitive count, curve stability, and layer organization.

Reconstruction-based supervision, commonly used in dataset-trained methods, encour-

ages models to reproduce observed examples as closely as possible. When paired with sequential or latent representations, this often leads to stable and coherent geometry within the distribution of the training data. For instance, models trained on icon datasets tend to produce clean paths and consistent decomposition because the supervision signal implicitly encodes structural regularities present in the dataset. However, this form of supervision can limit generalization, as geometry quality may degrade when models encounter scenes outside the training distribution.

Optimization-based supervision introduces a different geometric behavior. Methods that rely on pixel reconstruction losses optimize SVG parameters directly against a target image. This encourages accurate boundary alignment but does not penalize redundant primitives. As a result, geometry may become fragmented as additional paths are introduced to reduce visual error. The analysis of vectorization approaches suggests that pixel-driven supervision prioritizes local accuracy over global structure, which explains why visually precise results may still exhibit cluttered SVG representations.

Semantic supervision, particularly through CLIP guidance, shifts the focus from pixel-level accuracy to concept-level alignment. CLIP-based signals provide gradients that encourage the rendered SVG to match textual semantics rather than exact geometry. This often produces stable global composition but leaves fine geometric details unconstrained. Consequently, CLIP-guided methods may capture object identity while relying on additional optimization steps to refine boundaries and path quality.

Diffusion-based supervision further amplifies this semantic influence by providing strong distributional priors. Through SDS or related techniques, diffusion models guide optimization toward visually plausible images consistent with the prompt. This enables text-to-SVG generation without paired vector data, but it also introduces characteristic geometric effects. Because diffusion supervision operates in raster space and prioritizes realism, optimization may increase primitive count to approximate high-frequency details. This behavior explains why diffusion-guided approaches frequently achieve strong visual results while producing structurally dense SVGs.

Hybrid supervision strategies attempt to balance these effects by combining semantic guidance with structural constraints or learned representations. Models that integrate layered representations, neural initialization, or latent path spaces often exhibit more stable geometry because supervision acts within a constrained representation domain. Rather than freely adding primitives, optimization is restricted to a predefined structural space, which can improve interpretability and reduce redundancy.

Across methods, a consistent pattern emerges: supervision determines how aggressively geometry is refined. Pixel-based supervision promotes accuracy but may fragment structure. Semantic and diffusion-based supervision promote realism but may increase complexity. Dataset-based supervision promotes coherence within known distributions. Hybrid strategies introduce constraints that help regulate geometric behavior.

These observations highlight that geometry quality cannot be attributed to representation alone. Instead, structural outcomes emerge from the interaction between representation and supervision. Understanding this interaction is essential for explaining why models with similar visual performance may differ substantially in editability and structural cleanliness.

The next section builds on this interaction by examining a central trade-off observed across methods: the tension between visual realism and editability.

8.3 Editability versus Visual Realism Trade-off

A central pattern emerging from the comparative analysis is the tension between visual realism and editability. While many recent methods aim to maximize visual quality, the resulting SVGs do not always preserve the properties required for practical editing. This reveals that realism and editability are related but fundamentally distinct objectives.

Methods driven by strong semantic or diffusion-based supervision often achieve visually convincing outputs by refining geometry until the rendered image aligns with learned image priors. However, this refinement frequently increases structural complexity in ways that undermine usability. This tension is not merely theoretical: LIVE, following the optimization protocol also adopted by VectorFusion, represents a single image using up to 160 paths distributed across multiple layers [xingSVGDreamer2024]. At this scale, individual path manipulation becomes impractical in standard design tools such as Figma or Illustrator. Objects become entangled across paths, meaning that selecting or modifying a single visual component requires identifying and adjusting dozens of overlapping primitives. SVGDreamer++ explicitly acknowledges this failure mode in earlier methods, noting that optimization-based approaches produce outputs where objects are entangled across paths, making component-level editing effectively impossible [xingSVGDreamerpp2025].

It should be noted that this structural cost comes with a genuine visual benefit. Optimization-based methods, particularly VectorFusion [jainVectorFusion2022] and SVGDreamer [xingSVGDreamer2024], consistently produce the most visually rich and semantically detailed outputs among the methods analyzed in this thesis. The same mechanism that fragments structure — unconstrained primitive growth guided by strong diffusion priors — is precisely what enables photorealistic rendering quality. This confirms that the trade-off is genuine: neither side is simply better, and the appropriate choice depends on whether the intended use prioritizes visual output quality or downstream editability.

This behavior follows directly from how optimization-based supervision operates. By continuously adjusting parameters to match semantic guidance, these approaches in-

introduce many small primitives that capture texture, shading, or fine boundary detail. Although the rendered result appears realistic, the underlying SVG reflects optimization dynamics rather than design logic. The structure is shaped by what minimizes the loss, not by what makes the graphic interpretable or modifiable.

In contrast, models that explicitly constrain representation tend to produce outputs that are easier to edit, even when visual abstraction is introduced. Implicit layered approaches, for example, typically limit the number of shapes and enforce decomposition into interpretable components. As demonstrated in NeuralSVG [polaczekNeuralSVG2025], complex scenes can be represented using a small fixed number of layers, which improves usability despite reduced photorealistic detail. Similarly, dataset-trained icon generation models such as DeepSVG [carlierDeepSVG2020] and DeepIcon [bingDeepIcon2024] inherit structural simplicity from their training data, producing outputs that are more directly editable but constrained to the style distribution of the dataset.

LayerTracer [33] represents a different route to the same goal. Rather than constraining an optimization process, the model is trained directly on layered construction sequences that reflect how designers build SVG graphics step by step. As a result, editability is not an emergent property but a learned one — the model produces layered outputs because it was explicitly supervised to construct graphics in that manner, making it structurally closer to designer workflows than any optimization-based approach analyzed in this thesis.

The analysis suggests that the realism–editability trade-off is not accidental but emerges from how models balance supervision strength against representational constraints. When supervision strongly prioritizes visual similarity without structural regularization, primitive counts tend to increase and editability degrades. When representation enforces decomposition or limits complexity, editability improves but visual detail is reduced.

Importantly, this trade-off should not be interpreted as a strict opposition. Hybrid approaches suggest that combining semantic guidance with constrained representations can move toward both objectives simultaneously. Methods that integrate structural priors, layered representations, or latent constraints into the generation process appear most promising for bridging the gap between visual realism and practical usability. Recognizing this distinction helps explain why no single approach consistently dominates across evaluation dimensions, and why different methods are suited to different application contexts.

8.4 Complexity vs Control: Computational Implications

Beyond structural and visual properties, SVG generation methods differ significantly in computational behavior. The analysis indicates that the level of control a model provides over geometry and structure is closely related to its computational cost. This introduces another recurring trade-off: approaches that offer fine-grained control over primitives often require iterative optimization, while methods that generate outputs in a forward pass typically sacrifice direct control.

Optimization-based methods illustrate the high-control regime. Because these approaches update vector parameters iteratively, they allow precise adjustments to geometry and enable flexible integration of supervision signals. Designers and researchers can influence primitive count, initialization, or structural constraints during generation. However, this flexibility comes at the cost of inference time and computational resources. Iterative refinement, diffusion guidance, and repeated rasterization steps increase runtime, making such methods less suitable for interactive workflows without additional acceleration.

Implicit neural representations introduce a related but distinct computational profile. Although these models also rely on optimization, part of the structural reasoning is embedded in neural parameter space. This can reduce the need for large numbers of explicit primitives but may shift complexity toward evaluating neural functions and performing gradient updates. As a result, generation may remain computationally expensive even when outputs appear structurally compact. The analysis suggests that implicit approaches trade primitive complexity for optimization complexity.

Dataset-trained models occupy a different point in this spectrum. By learning structural patterns during training, they enable faster inference at generation time. Forward decoding eliminates iterative parameter updates, which improves efficiency and reproducibility. However, this shift reduces per-instance controllability. Adjusting structure after training may require latent manipulation rather than direct geometric editing, which can limit fine-grained control.

System-oriented pipelines further illustrate this balance between efficiency and control. Approaches that incorporate segmentation, layered reasoning, or diffusion transformers often distribute complexity across multiple stages. While this can improve structural quality and automation, it may also introduce dependencies on pretrained components and specialized hardware. Consequently, computational feasibility becomes an important factor when comparing methods beyond visual performance.

Across these approaches, a consistent pattern emerges: increased control over primitives and structure typically requires iterative optimization, while faster generation often relies on learned structural priors. This relationship highlights that computational complexity is not merely an implementation detail but a characteristic linked to representation and

supervision choices.

Recognizing this connection is important when interpreting evaluation results. A method producing highly editable outputs may incur higher computational cost, while a fast generation pipeline may rely on stronger structural assumptions learned from data. Understanding these trade-offs helps position methods within practical design contexts and clarifies why different approaches target different usage scenarios.

The next section builds on these observations to synthesize the key conditions that tend to produce clean and editable SVG graphics across model families.

8.5 Structurally Clean and Editable SVG Generation

Bringing together the previous analyses, a central question emerges: what conditions consistently lead to clean and editable SVG outputs? Rather than attributing this property to a single model family, the comparative perspective suggests that editability arises from a combination of representational constraints, supervision design, and structural priors.

A first recurring condition is explicit or implicit control over primitive count. Methods that allow primitives to grow freely during optimization often produce visually detailed but structurally dense SVGs. In contrast, approaches that limit the number of shapes, enforce layered decomposition, or operate within constrained latent spaces tend to produce outputs that are easier to interpret. This suggests that primitive budgeting functions as an important structural regularizer, even when it is not explicitly framed as an evaluation objective.

A second condition concerns decomposition aligned with semantic structure. SVGs become more editable when objects correspond to meaningful components rather than arbitrary geometric fragments. Layered representations, neural scene decomposition, and designer-inspired pipelines often encourage this alignment. The resulting graphics exhibit clearer separation between elements, which supports localized edits such as recoloring, repositioning, or shape refinement.

Another important factor is the domain in which supervision is applied. When supervision operates exclusively in raster space, optimization may prioritize pixel-level accuracy without preserving geometric simplicity. In contrast, approaches that introduce supervision within constrained representations — for example through latent path spaces or neural layer parameterization — tend to regulate how geometry evolves during generation. This suggests that supervision domain influences not only realism but also structural stability.

Initialization strategies also appear to affect editability. Random initialization combined with unconstrained optimization can lead to unstable structure, while informed initialization — such as segmentation, saliency, or learned latent embeddings — provides a starting point closer to meaningful decomposition. This reduces the need for excessive primitive refinement and can improve structural coherence.

Finally, architectural bias toward design logic plays an increasingly visible role. Methods that incorporate notions of layering, sequential construction, or scene composition often produce outputs that resemble human design workflows. While these approaches do not guarantee minimal structure, they introduce priors that favor interpretability. The presence of such priors suggests that editability is partly an architectural property rather than solely an optimization outcome.

Taken together, these observations indicate that clean and editable SVG generation emerges when models balance flexibility with structural constraints. Primitive count control, semantically aligned decomposition, supervision within structured representation spaces, informed initialization, and design-aware architectural choices all contribute to this outcome.

Importantly, these conditions are not exclusive to a single method category. Instead, they represent recurring patterns across model families. Recognizing these patterns provides a foundation for understanding why certain approaches produce SVGs that are not only visually convincing but also practically usable.

The following section concludes the synthesis by translating these observations into broader design implications for future SVG generation research.

8.6 Design Implications for Future SVG Generation

The cross-model analysis presented in this chapter suggests that progress in SVG generation is not determined solely by improvements in visual realism, but by how models integrate structural reasoning into the generation process. Rather than identifying a single dominant paradigm, the synthesis highlights several design implications that may guide future research.

First, representation should be treated as a structural decision rather than only a modeling choice. The findings indicate that representations which constrain decomposition — such as layered, hierarchical, or latent path-based formulations — tend to produce outputs that are easier to interpret and modify. This suggests that future models may benefit from incorporating structural priors directly into representation design, instead of relying on optimization dynamics to discover structure implicitly.

Second, supervision strategies should balance semantic guidance with structural regularization. Strong raster-space supervision, particularly from diffusion models, enables realistic outputs but may encourage geometric complexity. Introducing constraints within the representation domain — for example through limited primitive budgets, layer consistency objectives, or path-level latent spaces — appears to moderate this effect. This indicates that supervision design plays a key role not only in realism but also in structural quality.

Third, evaluation practices may need to expand beyond visual metrics. The multi-dimensional framework proposed earlier emphasizes that editability, structural cleanliness, and geometric stability are central properties of vector graphics. Future benchmarks that explicitly measure these dimensions could enable more consistent comparison across methods and reduce the current reliance on qualitative inspection.

Another implication concerns the integration of design logic into generation pipelines. Several recent approaches move toward modeling construction processes rather than final appearance alone. This trend suggests that incorporating designer-inspired decomposition, layering strategies, or sequential construction priors may provide a pathway toward SVG outputs that are both realistic and editable.

Finally, the relationship between computational complexity and controllability highlights an important practical consideration. Methods that offer fine-grained control often require iterative optimization, while efficient forward models depend on learned structural priors. Future systems may therefore focus on hybrid designs that combine fast generation with mechanisms for controllable refinement, enabling both scalability and usability.

Overall, these implications reinforce a central conclusion of this thesis: generating high-quality SVG graphics requires aligning representation, supervision, and evaluation with the structural nature of vector graphics. Approaches that treat SVG generation as a structured design problem — rather than solely an image synthesis task — appear best positioned to advance the field.

The next chapter concludes the thesis by summarizing the main findings, reflecting on limitations, and outlining directions for future research.

9 Conclusion

This thesis examined SVG generation and vectorization methods from the perspective of representation, supervision, and structural quality. While recent progress in generative modeling has significantly advanced raster image synthesis, vector graphics generation introduces additional challenges due to the structured nature of SVG. Unlike raster outputs, SVG graphics must preserve geometry, hierarchy, and editability, which makes evaluation and model design fundamentally different from conventional image generation tasks.

Through a comparative analysis of representative models, this work showed that the internal representation used by a method strongly influences the structure of the resulting SVG. Parametric optimization approaches provide direct geometric control but often produce redundant paths when supervision operates primarily in raster space. Sequential and dataset-trained models can generate coherent structure within constrained domains, while implicit and layered neural representations introduce architectural constraints that encourage compact and interpretable decomposition. These observations indicate that structural quality is not solely an optimization outcome but emerges from representational design.

The analysis further demonstrated that supervision signals shape geometric behavior in complementary ways. Pixel-based reconstruction encourages accurate boundary matching but may fragment structure, whereas semantic and diffusion-based supervision promote visual realism while increasing primitive complexity. Hybrid approaches that combine semantic guidance with constrained representations appear to balance these effects more effectively, suggesting that supervision design plays a central role in regulating structural outcomes.

A recurring insight across model families is the presence of a trade-off between visual realism and editability. Methods optimized for photorealistic appearance often introduce structural complexity that reduces usability, while approaches that enforce layered or constrained representations tend to produce outputs that are easier to modify. This trade-off highlights that SVG generation quality cannot be evaluated using visual metrics alone and must consider structural properties that affect downstream design workflows.

To address this gap, the thesis proposed a multi-dimensional perspective on SVG evalu-

ation. Rather than relying on a single metric, SVG quality should be considered across semantic alignment, structural cleanliness, editability, geometric stability, and computational practicality. This perspective clarifies why methods with similar visual performance may differ substantially in usability and provides a consistent lens for comparing approaches across representation and supervision choices.

Based on the synthesis of findings, several conditions were identified as recurring contributors to clean and editable SVG generation. These include constraints on primitive count, decomposition aligned with semantic structure, supervision applied within structured representation spaces, informed initialization strategies, and architectural priors that reflect design logic. Importantly, these conditions appear across different model categories, suggesting that editability emerges from coordinated design decisions rather than a single methodological paradigm.

Overall, this thesis contributes an analytical understanding of how SVG generation methods construct vector graphics and why structural outcomes differ across approaches. By framing SVG generation as a structured design problem rather than purely an image synthesis task, the work highlights the importance of aligning representation, supervision, and evaluation with the properties that make vector graphics useful in practice.

Future research may explore evaluation protocols that explicitly measure editability and structural quality, develop hybrid architectures that combine fast generation with controllable refinement, and integrate designer-oriented priors into generative pipelines. Advancing SVG generation will likely depend not only on improving visual realism but on developing models that produce vector graphics that remain interpretable, efficient, and practically usable.

Bibliography

- [1] J. David Eisenberg. *SVG Essentials: Producing Scalable Vector Graphics with XML*. Repr. Beijing Köln: O'Reilly, 20.
- [2] A. Quint. “Scalable Vector Graphics”. In: *IEEE Multimedia* 10.3 (July 2003), pp. 99–102.
- [3] Xu Ma, Yuqian Zhou, Xingqian Xu, Bin Sun, Valerii Filev, Nikita Orlov, Yun Fu, Humphrey Shi. *Towards Layer-wise Image Vectorization*. June 2022.
- [4] Alexandre Carlier, Martin Danelljan, Alexandre Alahi, Radu Timofte. “DeepSVG: A Hierarchical Generative Network for Vector Graphics Animation”. In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, H. Lin. Vol. 33. Curran Associates, Inc., 2020, pp. 16351–16361.
- [5] Vikas Thamizharasan, Difan Liu, Matthew Fisher, Nanxuan Zhao, Evangelos Kalogerakis, Michal Lukac. *NIVeL: Neural Implicit Vector Layers for Text-to-Vector Generation*. May 2024.
- [6] Diederik P. Kingma, Max Welling. *Auto-Encoding Variational Bayes*. Dec. 2022.
- [7] Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, Jonathan Ragan-Kelley. “Differentiable Vector Graphics Rasterization for Editing and Learning”. In: *ACM Trans. Graph.* 39.6 (Nov. 2020), 193:1–193:15.
- [8] Ian Goodfellow, Yoshua Bengio, Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [9] Justin Johnson, Alexandre Alahi, Li Fei-Fei. *Perceptual Losses for Real-Time Style Transfer and Super-Resolution*. 2016.
- [10] Karen Simonyan, Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition”. In: *ICLR*. 2015.
- [11] Mark Everingham et al. “The PASCAL Visual Object Classes (VOC) Challenge”. In: *IJCV*. 2010.
- [12] Alec Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. Feb. 2021.
- [13] Jonathan Ho, Ajay Jain, Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. In: *NeurIPS*. 2020.
- [14] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, Björn Ommer. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2021.

- [15] Ben Poole, Ajay Jain, Jonathan T. Barron, Ben Mildenhall. *DreamFusion: Text-to-3D Using 2D Diffusion*. Sept. 2022.
- [16] Ajay Jain, Amber Xie, Pieter Abbeel. *VectorFusion: Text-to-SVG by Abstracting Pixel-Based Diffusion Models*. Nov. 2022.
- [17] Ximing Xing, Haitao Zhou, Chuang Wang, Jing Zhang, Dong Xu, Qian Yu. *SVG-Dreamer: Text Guided SVG Generation with Diffusion Model*. Dec. 2024.
- [18] David Ha, Douglas Eck. *A Neural Representation of Sketch Drawings*. May 2017.
- [19] Leo Sampaio Ferraz Ribeiro, Tu Bui, John Collomosse, Moacir Ponti. *Sketch-former: Transformer-based Representation for Sketched Structure*. Feb. 2020.
- [20] Yaroslav Ganin, Tejas Kulkarni, Igor Babuschkin, S. M. Ali Eslami, Oriol Vinyals. *Synthesizing Programs for Images Using Reinforced Adversarial Learning*. Apr. 2018.
- [21] Raphael Gontijo Lopes, David Ha, Douglas Eck, Jonathon Shlens. *A Learned Representation for Scalable Vector Graphics*. Apr. 2019.
- [22] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, Ren Ng. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. Aug. 2020.
- [23] Juan A. Rodriguez, Abhay Puri, Shubham Agarwal, Issam H. Laradji, Pau Rodriguez, Sai Rajeswar, David Vazquez, Christopher Pal, Marco Pedersoli. *StarVector: Generating Scalable Vector Graphics Code from Images and Text*. May 2025.
- [24] Yiyang Yang, Wei Cheng, Sijin Chen, Xianfang Zeng, Fukun Yin, Jiaxu Zhang, Liao Wang, Gang Yu, Xingjun Ma, Yu-Gang Jiang. *OmniSVG: A Unified Scalable Vector Graphics Generation Model*. May 2025.
- [25] Ximing Xing, Juncheng Hu, Guotao Liang, Jing Zhang, Dong Xu, Qian Yu. *Empowering LLMs to Understand and Generate Complex Vector Graphics*. Mar. 2025.
- [26] Haomin Wang et al. *InternSVG: Towards Unified SVG Tasks with Multimodal Large Language Models*. Oct. 2025.
- [27] Haokun Zhu, Juang Ian Chong, Teng Hu, Ran Yi, Yu-Kun Lai, Paul L. Rosin. *SAMVG: A Multi-stage Image Vectorization Model with the Segment-Anything Model*. Dec. 2023.
- [28] Alexander Kirillov et al. *Segment Anything*. Apr. 2023.
- [29] Pradyumna Reddy, Michael Gharbi, Michal Lukac, Niloy J. Mitra. *Im2Vec: Synthesizing Vector Graphics without Vector Supervision*. Apr. 2021.
- [30] Ximing Xing, Qian Yu, Chuang Wang, Haitao Zhou, Jing Zhang, Dong Xu. “SVG-Dreamer++: Advancing Editability and Diversity in Text-Guided SVG Generation”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47.7 (July 2025), pp. 5397–5413.
- [31] Sagi Polaczek, Yuval Alaluf, Elad Richardson, Yael Vinker, Daniel Cohen-Or. *NeuralSVG: An Implicit Representation for Text-to-Vector Generation*. 2025.

- [32] Qi Bing, Chaoyi Zhang, Weidong Cai. *DeepIcon: A Hierarchical Network for Layer-wise Icon Vectorization*. Oct. 2024.
- [33] Yiren Song, Danze Chen, Mike Zheng Shou. *LayerTracer: Cognitive-Aligned Layered SVG Synthesis via Diffusion Transformer*. Aug. 2025.
- [34] Peiying Zhang, Nanxuan Zhao, Jing Liao. *Text-to-Vector Generation with Neural Path Representation*. May 2024.
- [35] Ximing Xing, Juncheng Hu, Jing Zhang, Dong Xu, Qian Yu. *SVGFusion: Scalable Text-to-SVG Generation via Vector Space Diffusion*. Mar. 2025.

List of Figures

Appendix

Great table.

Headline 1	Headline 2
Titel 1	Description 1
Titel 2	Description 2

Table 1: Overview in a great table.

Table of chosen approaches

Inference Time Overview

Model	Task Type	Inference	Input	Output	Supervision	Core Idea
VectorFusion	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup	Distills diffusion priors via differentiable rasterization.
SVGFusion	Text-to-SVG	Forward Dec.	Text	Paths	Supervised	Continuous latent space learning using SVGX dataset.
SVGDreamer	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup	Semantic-driven vectorization (SIVE) and Particle SDS.
SVGDreamer++	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup	Hierarchical optimization with adaptive primitive control.
T2V-NPR	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup	Dual-branch VAE learns path latent space constraints.
LayerTracer	System/Pipeline	Hybrid	Text/Img	Layers/Paths	Supervised	Models designer construction logic via DiT sequences.
Im2Vec	Vectorization	Forward Dec.	Img	Paths	Self-Sup	Differentiable rasterizer allows learning from appearance.
SAMVG	Vectorization	Iterative Opt.	Image	Paths	Opt.-based	Uses SAM for semantic initialization in vectorization.
NIVeL	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup	Implicit neural layers as optimization domain for SDS.
NeuralSVG	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup	Scene encoded in MLP weights for layered shapes.
LIVE	Vectorization	Iterative Opt.	Image	Layers/Paths	Opt.-based	Layer-wise hierarchical optimization of vector graphs.
DeepSVG	Generation	Forward Dec.	None	Paths	Supervised	Hierarchical Transformer for shape/command disentanglement.
DeepIcon	Vectorization	Forward Dec.	Image	Layers/Paths	Supervised	Hierarchical network for direct image-to-SVG translation.

Training Architecture – Optimization-Based Methods

Model	Trainables	Training Data	Init.	Supervision	Representation	Priors
LIVE	SVG parameters	None	Raster-driven	None	Parametric (flat)	None
SAMVG	None	None	Mask-based	None	Parametric (mask-derived)	SAM
VectorFusion	SVG parameters	None	Random / Trace	SDS (Stable Diff.)	Parametric (flat)	Stable Diff.
SVGDreamer	SVG parameters / LoRA	None	Random paths	VPD	Parametric (flat)	Stable Diff.
SVGDream++	SVG parameters / LoRA	None	Random + HIVE	VPD + HIVE loss	Structured parametric	Stable Diff. + SAM
NeuralSVG	MLP weights + colors/ LoRA	None	Saliency-based	SDS	Implicit neural fields	Stable Diff.
NIVeL	MLP weights + layer colors	None	Random weights	SDS	Implicit neural fields	DeepFloyd + CLIP

Training Architecture – Dataset-Trained Methods

Model	Trainables	Training Data	Init.	Supervision	Representation	Priors
DeepSVG	VAE (Transformer) weights	SVGIcons8	Latent sampling	Recon. + KL	Sequential path tokens	None
DeepIcon	Transformer weights	SVGIcons8	CLIP embedding	Reconstruction	Sequential path tokens	CLIP encoder
SVGFusion	VP-VAE + VS-DiT	SVGX	Random latent	Recon. + Denoising	Continuous latent space	CLIP + DINO-v2
T2V-NPR	NPR-VAE + Latents + LoRA	FIGR-8	Random path latents	VSD	Neural Path Repr.	Stable Diff. + CLIP
Im2Vec	Encoder + RNN + 1D CNN	Raster images	Image encoding	Reconstruction	Parametric paths	None
LayerTracer	DiT + LoRA	Serpentine dataset	Diffusion noise	Denoising loss	Layered pixel sequences	Flux DiT + Florence-2

Note: - Order in table: non-learning -i optimization-based -i implicit -i dataset-trained -i systems.

- For optimization-based methods, “trainables” means to per-instance SVG parameters optimized at inference time rather than learned model weights.

- Trained on text prompts means, optimize SVG params at inference time using a frozen pretrained model as semantic prior.

- Semantic distillation means supervision signal got by backpropagating semantic alignments scores from Stable Diff. model to optimize target representation.

- SVGFusion, first trains VAE on SVGX dataset to learn a latent representations of SVGs, then diffusion transformer is trained on those latent repr. + noise. The goal of diff. transformer is to learn to remove the noise. -i if a model learns how to denoise the latent vector, it learns also how to generate new latent vectors.

- A parametric internal representation models SVG graphics as continuous parameters (e.g., Bezier control points and colors) that can be directly optimized with gradients.

- Coarse-to-fine refinement starts from a rough initial set of shapes and then gradually adds paths, repeatedly checking and adjusting them to better match the target description.

- T2V w/NPR: They trained the Neural Path Representation (dual-branch VAE) on the FIGR-8 SVG dataset and it learns: how vector paths map to a latent path space and how to decode latents back into valid vector paths. VSD extends SDS by projecting diffusion-based pixel gradients into the neural path latent space for more stable vector optimization.

- DeepSVG is used by sampling or modifying latent vectors: each random sample produces a new SVG icon, and by interpolating or editing these latent vectors, users can generate variations or blends of existing icons.

Reproducibility Overview

Model	Code	Pre-trained	Hardware	Cost of time	Reproduction Feasibility
LIVE	Yes	No	RTX 3090	2,600–10,000 s	Feasible on high-end consumer GPU (slow).
SAMVG	No	No	RTX 3090	139–2,000 s	Re-implementation possible but requires effort.
VectorFusion	No	No	Not specified	Unknown	Not reproducible due to lack of public implementation.
SVGDreamer	Yes	No	V100 (≥ 31 GB VRAM)	7–14 min	Requires high-VRAM GPU and LoRA pre-training.
SVGDreamer++	Yes	No	A800 (≥ 31 GB VRAM)	7 min	Requires high-VRAM GPU and LoRA pre-training.
NeuralSVG	Yes	No	Not stated	Unknown	Hardware requirements unclear.
NIVeL	Pending	No	A100	5 min	High-end GPU required.
T2V-NPR	Yes	VAE (unknown)	RTX 3090	13 min	Feasible on high-end consumer GPU.
DeepSVG	Yes	VAE (unknown)	2×1080Ti (training)	Unknown	Feasible on consumer GPU.
DeepIcon	No	Transformer (unknown)	Not specified	–	Not reproducible (no official code).
SVGFusion	No	VP-VAE + VS-DiT (unknown)	Multi-A800	24–36 s	No official implementation available.
Im2Vec	Yes	VAE (unknown)	Not specified	Unknown	Likely feasible on consumer GPU.
LayerTracer	Yes	DiT + LoRA (unknown)	A100 (80GB)	27–34 s	Enterprise GPU required.

Declaration on oath

I hereby certify that I have written my master thesis independently and have not yet submitted it for examination purposes elsewhere. All sources and aids used are listed, literal and meaningful quotations have been marked as such.

March 10, 2026, Your Name

Consent to plagiarism check

I hereby agree that my submitted work may be sent to PlagAware (<https://my.plagaware.com/>) in digital form for the purpose of checking for plagiarism and that it may be temporarily (max. 5 years) stored in the database maintained by PlagScan as well as personal data which are part of this work may be stored there.

Consent is voluntary. Without this consent, the plagiarism check cannot be prevented by removing all personal data and protecting the copyright requirements. Consent to the storage and use of personal data may be revoked at any time by notifying the faculty.

March 10, 2026, Your Name